

Tutorial de MicroPython

Índice

Índice	1
1. Aprendizaje de la programación	2
1.1 Antes de empezar: Notas importantes	2
1.2 Instalar el IDE Thonny	2
1.3 Instalar el controlador USB CH340	5
1.4 Flashear el firmware de MicroPython mediante el IDE Thonny	8
1.5 Descripción general del IDE Thonny	12
1.6 Ejecutar tu primer código: ejecuta el código actual en Thonny	14
1.7 Carga de código en la placa ESP32: ejecutar código en ESP32	16
Método 1: Guardar el código como main.py (en el dispositivo ESP32)	16
Método 2: main.py llama a otros archivos	18
1.8 Ejemplos de código	23
1.8.1 Reproducir canciones	23
1.8.2 Cómo controlar un servo	24
1.8.3 Leer el valor del joystick	26
1.8.4 Robot controlado por joystick (con función de grabación y repetición de movimientos)	27
1.8.5 Leer el valor de la aplicación web	29
1.8.6 Brazo robótico controlado mediante la aplicación web	31
2. Flashear el firmware	34
2.1 Herramienta de grabación «flash_download_tool»	35
2.2 Firmware eArm de (firmware de fábrica)	36
2.2.1 Flashear el firmware del eArm con	36
2.3 Firmware de control mediante joystick « » (con función de grabación)	38
2.3.1 Grabar el firmware de control del joystick de	38
3. Problemas habituales y soluciones	41
3.1 ¿Cómo instalar el IDE Thonny en macOS?	41
3.2 El brazo robótico no funciona	44
3.3 Errores de conexión de puertos y reconocimiento de dispositivos	44
3.4 Errores de configuración del firmware y del intérprete de MicroPython	46
3.5 Errores de tiempo de ejecución (ejecución de código en ESP32)	48
3.6 Anomalías en el sistema de archivos y la visualización del dispositivo	52
3.7 Problemas de ocupación del puerto serie y de pérdida de conexión	53
4. Ponte en contacto con nosotros	54

Aprendizaje de la programación

1.1 Antes de empezar: Notas importantes

La placa de control ESP32-C3 viene con un firmware preinstalado que permite que el brazo robótico funcione tal y como se describe en la «**Guía de montaje y uso**». Este firmware permanecerá en la placa hasta que actualices el firmware de MicroPython mediante el IDE Thonny.

Nota: La actualización del firmware de MicroPython es totalmente reversible. Tras instalar MicroPython en tu placa ESP32-C3, puedes volver a Arduino en cualquier momento simplemente cargando un sketch de Arduino mediante el IDE de Arduino.

El paquete del tutorial incluye una recopilación de programas de ejemplo de MicroPython. Antes de continuar, descarga el paquete del tutorial y recuerda la carpeta en la que lo guardas, ya que necesitarás abrir estos archivos de ejemplo más adelante.

01_MicroPython > Example_Codes			
 buzzer	2026/1/6 17:47	Python file	4 KB
 hello_world	2026/1/7 11:55	Python file	1 KB
 joystick	2026/1/8 21:24	Python file	4 KB
 joystick_control_eArm	2026/3/26 22:43	Python file	15 KB
 main	2026/1/7 16:32	Python file	4 KB
 servo	2026/1/7 15:14	Python file	6 KB
 song	2026/1/6 17:46	Python file	7 KB
 web_app	2026/1/8 16:33	Python file	11 KB
 web_app_control_eArm	2026/1/9 8:56	Python file	16 KB

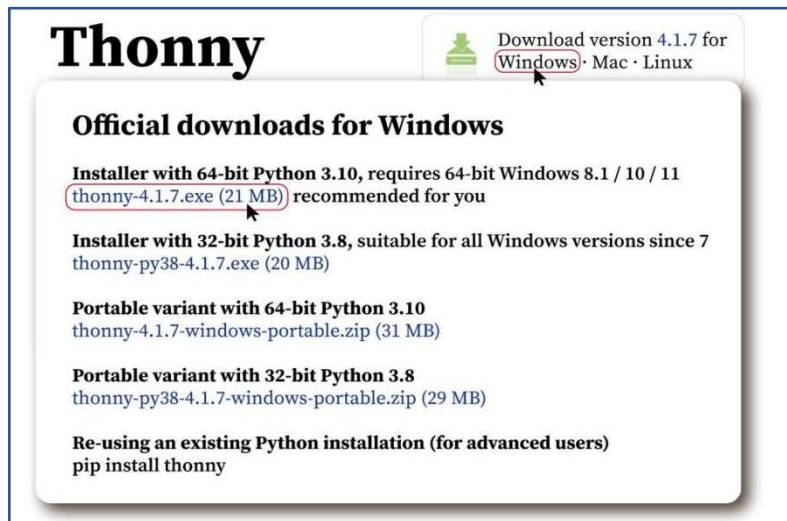
1.2 Instalar el IDE Thonny

Página web oficial de Thonny: <https://thonny.org>

Thonny es compatible con los tres sistemas operativos principales: Windows, macOS y Linux. Sigue las instrucciones correspondientes al sistema operativo que estés utilizando.

Sistema operativo	Métodos de instalación
Windows	I) Instalación del IDE de Thonny – PC con Windows
macOS	II) Instalación del IDE Thonny – Mac OS

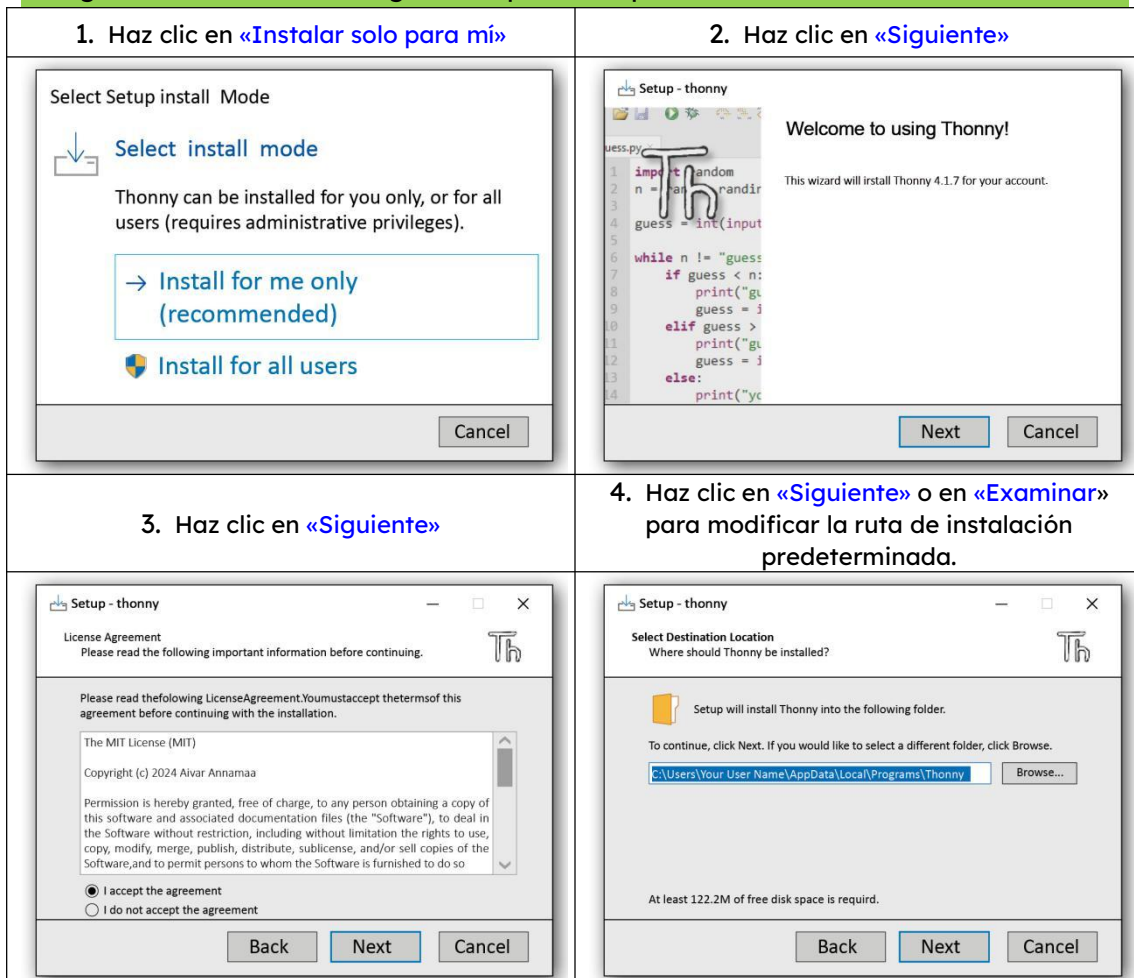
I) Instalación del IDE Thonny - PC con Windows

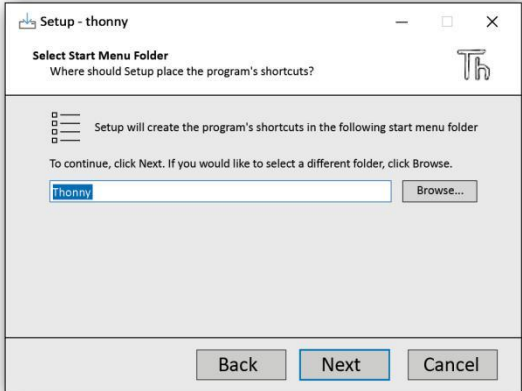
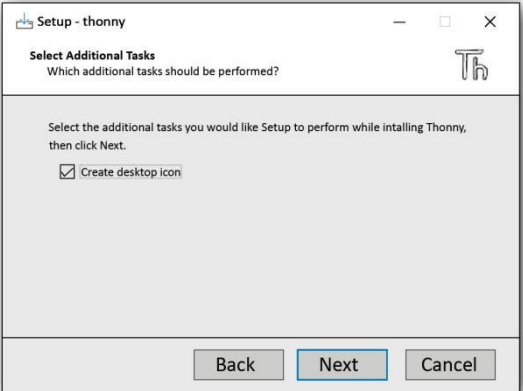
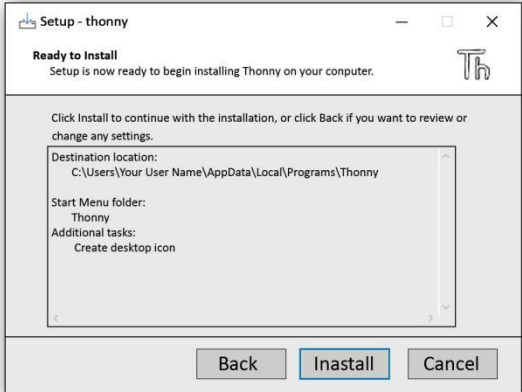
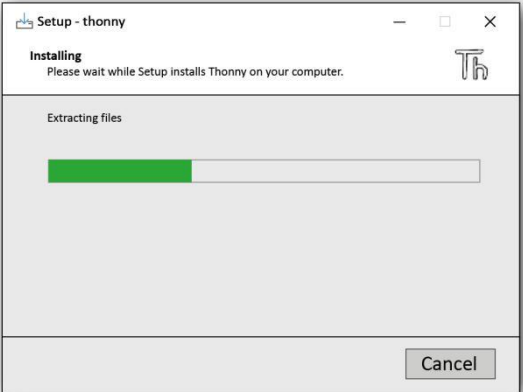
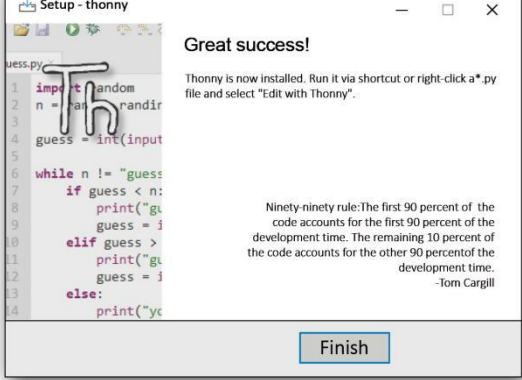


1. Descarga el instalador thonny-4.1.7.exe y haz doble clic para ejecutarlo.



2. Sigue haciendo clic en «Siguiente» para completar la instalación.



5. Haz clic en «Siguiente»	6. Marca la casilla «Crear icono en el escritorio» y haz clic en «Siguiente»
	
7. Haz clic en «Instalar»	8. Espera unos segundos a que finalice la instalación.
	
9. Finalizar	
	

3. Una vez finalizada la instalación, busca el icono de la aplicación en el escritorio para ejecutarla.

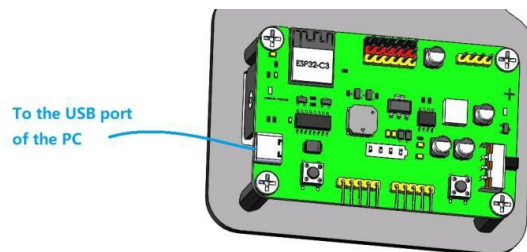


1.3 Instalar el controlador USB CH340

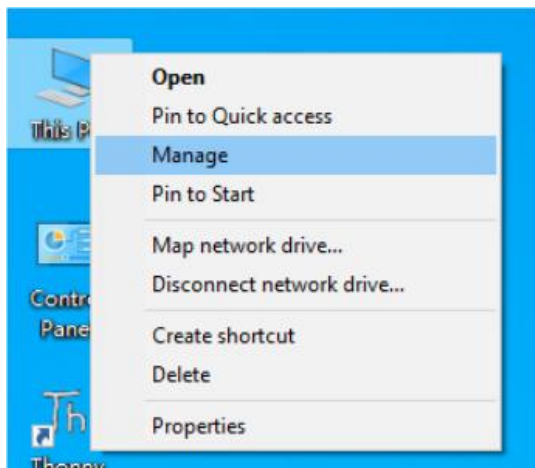
Sistema operativo	Métodos de instalación
Windows	I) Instalar el controlador en un sistema Windows
Linux	<u>II) Instalar el controlador en un sistema Linux</u>
Mac OS	<u>III) Instalar el controlador en un sistema Mac</u>

I) Instalar el controlador en un sistema Windows

1. Conecta tu ordenador y el ESP32 con un cable USB.

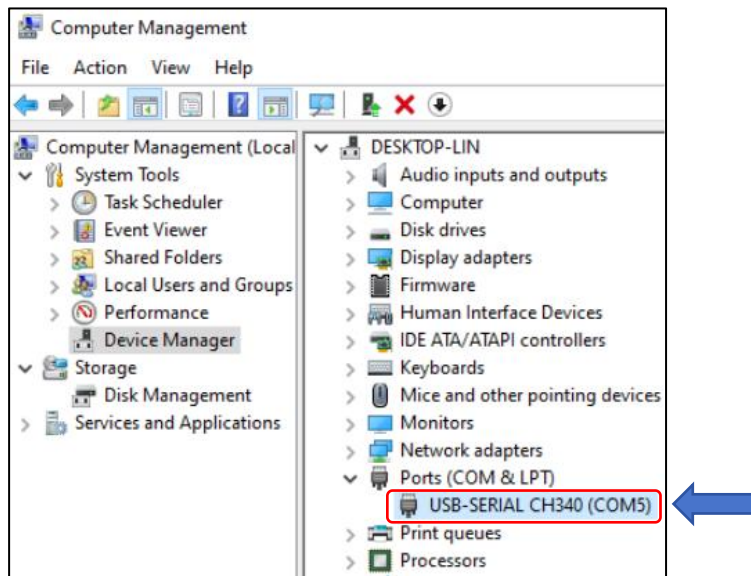


2. Vaya a la interfaz principal de su ordenador, seleccione «Este PC» y haga clic con el botón derecho del ratón para seleccionar «Administrar».



3. Comprueba si se ha instalado el controlador CH340

Si el controlador está correctamente instalado, cuando la placa ESP32 se conecta al ordenador mediante un cable USB, el dispositivo «USB-Serial CH340 (COMX)» aparecerá en la ruta «Administrador de dispositivos» → Puertos (COM y LPT), tal y como se muestra en la siguiente figura.

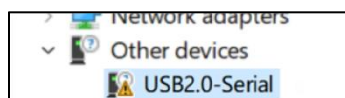


Nota: El nombre del dispositivo mostrado es «USB-SERIAL CH340 (COMx)», donde «x» puede ser cualquier número, ya que depende del número que tu ordenador asigne al dispositivo ESP32.

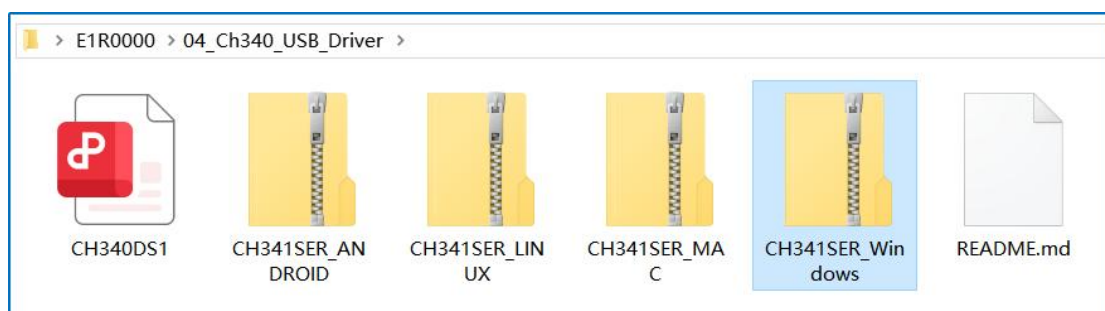
Si el controlador ya está instalado, omite el paso de instalación del controlador y pasa al siguiente capítulo.

4. Instalar el controlador CH340 manualmente

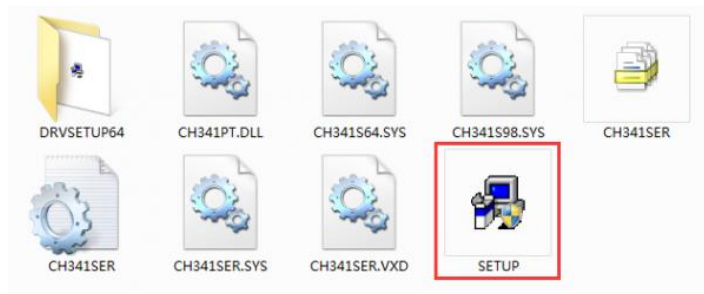
Si el controlador CH340 no está instalado, el Administrador de dispositivos de Windows suele mostrar un «USB2.0-Serial» o un «Dispositivo desconocido» con un signo de exclamación amarillo en «Puertos (COM y LPT)» o «Otros dispositivos». Esto indica que el ordenador ha detectado el hardware, pero no puede utilizarlo correctamente. Como resultado, no se asignará un número de puerto COM, lo que impedirá la comunicación serie.



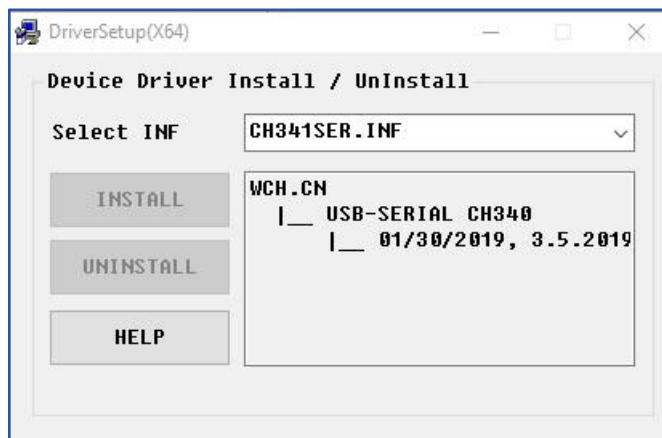
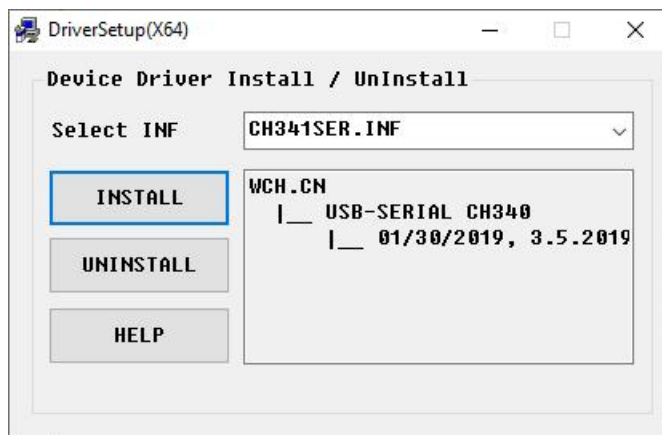
El archivo del controlador se incluye en el paquete del tutorial:



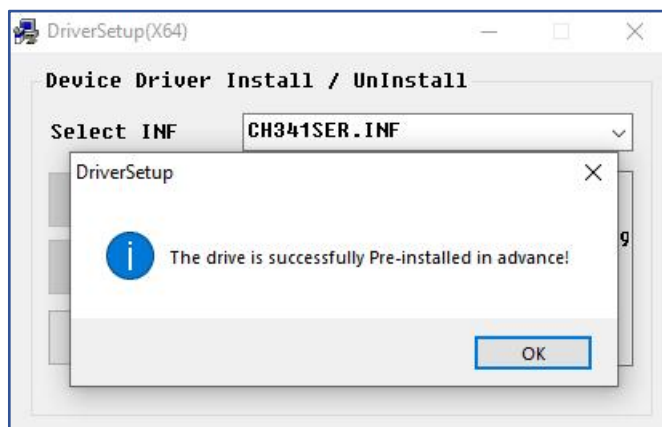
Descomprime el archivo que te hemos proporcionado y haz doble clic en «**SETUP**» para ejecutar el instalador:



Haz clic en «**INSTALL**» y espera a que finalice la instalación.



Instalación completada con éxito. Cierra todas las interfaces.

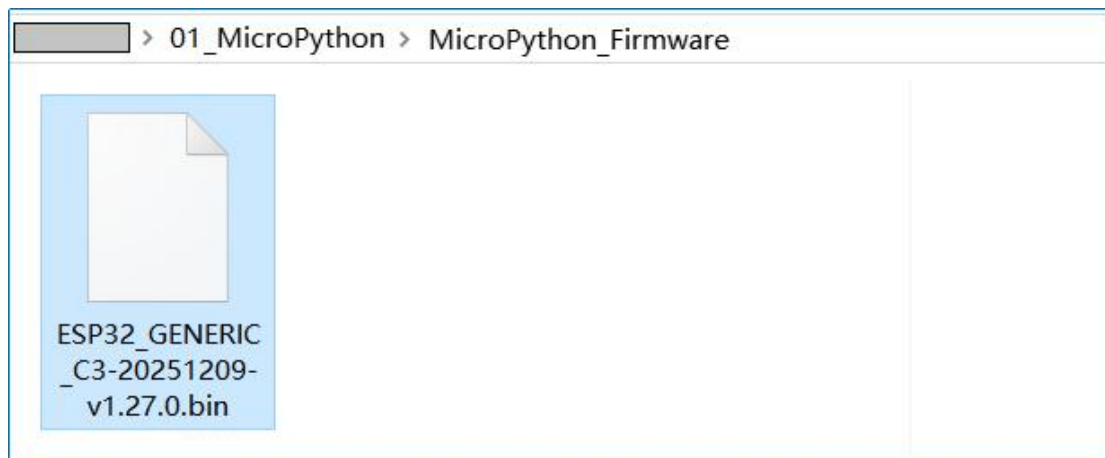


1.4 Flashear el firmware de MicroPython mediante el IDE Thonny

Para cargar código en un ESP32 desde el IDE Thonny, primero debes asegurarte de que el ESP32 tenga instalado el firmware de MicroPython y de que Thonny esté configurado para utilizar el intérprete de MicroPython.

El firmware utilizado en este tutorial es **ESP32_GENERIC_C3-20251209-v1.27.0.bin**

Este archivo de firmware de MicroPython se incluye en el paquete del tutorial:



O descárgalo de la página web oficial

Lista de firmware de MicroPython para ESP32-C3:

https://micropython.org/download/ESP32_GENERIC_C3/

Firmware

Releases

v1.27.0 (2025-12-09) .bin / [.app-bin] / [.elf and .map] / [Release notes] (latest)

v1.26.1 (2025-09-11) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.26.0 (2025-08-09) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.25.0 (2025-04-15) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.24.1 (2024-11-29) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.24.0 (2024-10-25) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.23.0 (2024-06-02) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.22.2 (2024-02-22) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.22.1 (2024-01-05) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.22.0 (2023-12-27) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.21.0 (2023-10-05) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.20.0 (2023-04-26) .bin / [.elf and .map] / [Release notes]

v1.19.1 (2022-06-18) .bin / [.elf and .map] / [Release notes]

v1.18 (2022-01-17) .bin / [.elf and .map] / [Release notes]

v1.17 (2021-09-02) .bin / [.elf and .map] / [Release notes]

Preview builds

These are automatic builds of the development branch for the next release.

v1.28.0-preview.121.g2b4b05a422 (2026-02-03) .bin / [.app-bin] / [.elf] / [.map]

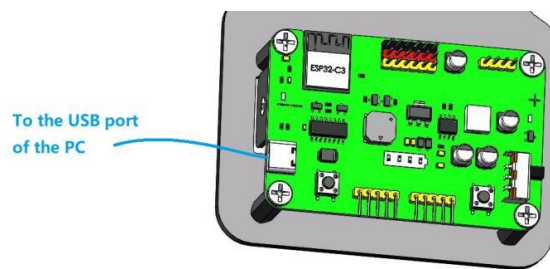
v1.28.0-preview.117.g023a49c55e (2026-01-31) .bin / [.app-bin] / [.elf] / [.map]

v1.28.0-preview.110.gaf76da96a3 (2026-01-29) .bin / [.app-bin] / [.elf] / [.map]

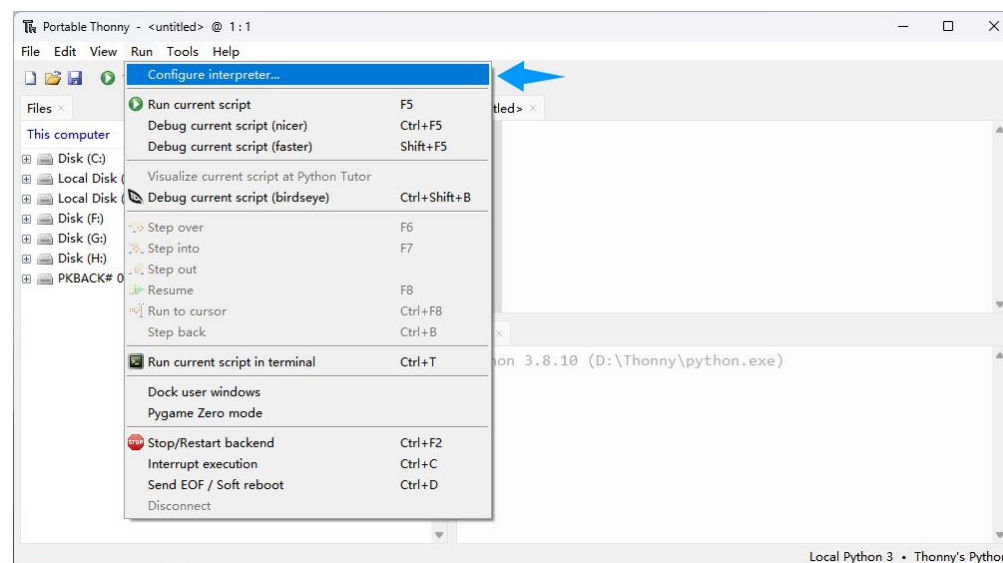
v1.28.0-preview.109.gbe15be3ab5 (2026-01-27) .bin / [.app-bin] / [.elf] / [.map]

Sigue los siguientes pasos:

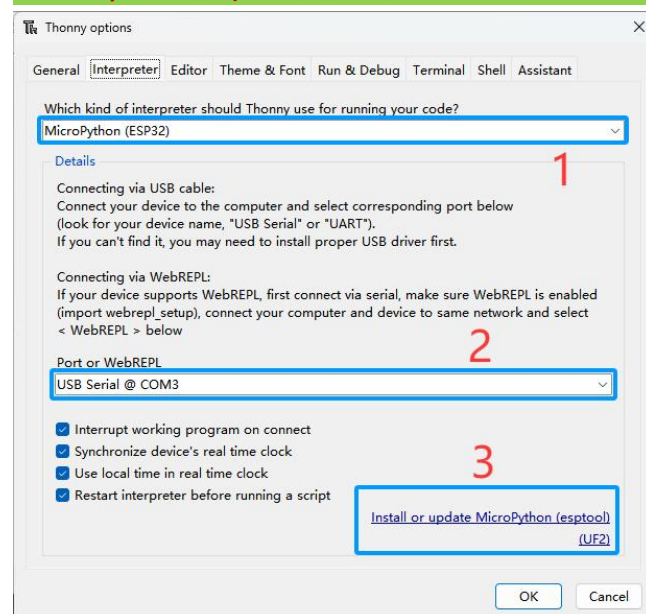
1) Conecta tu placa ESP32 al ordenador.



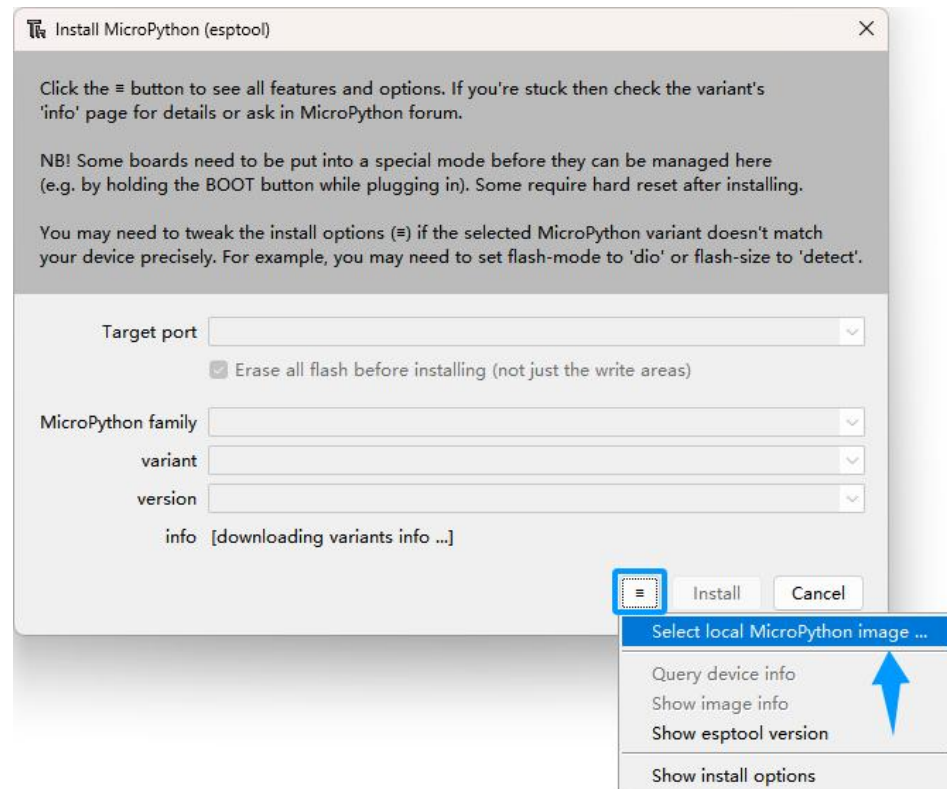
2) Abre el IDE Thonny. Haz clic en «Run» y selecciona «Configure interpreter...»



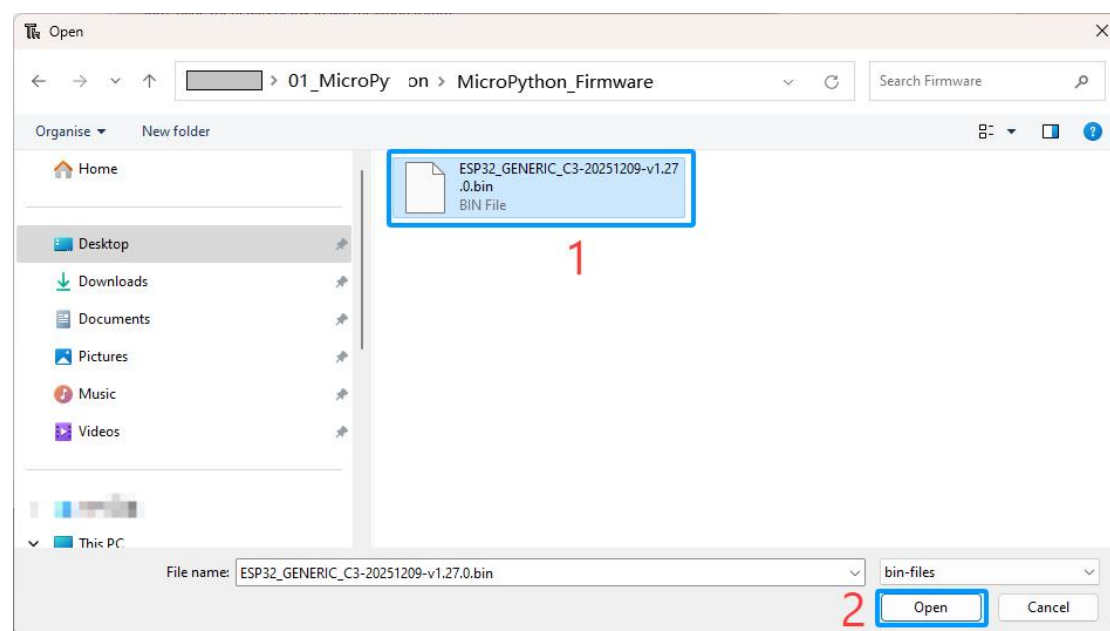
3) Selecciona «Micropython (ESP32)», selecciona «USB Serial @ COMX» (dependiendo del número asignado por el Administrador de dispositivos) y, a continuación, haz clic en el botón largo situado debajo de «Instalar o actualizar MicroPython (esptool) (UF2)».



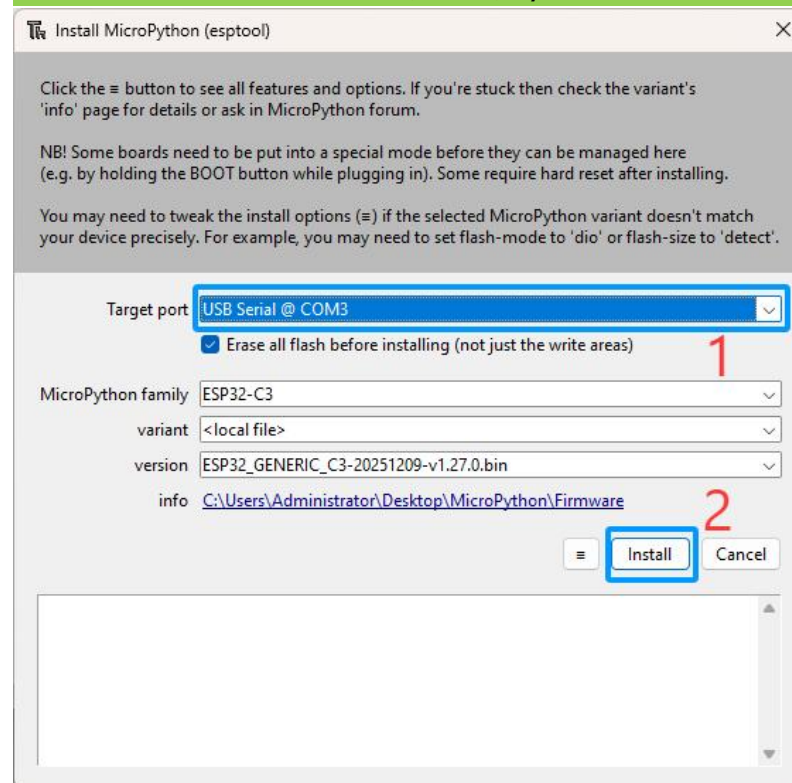
4) Aparecerá el siguiente cuadro de diálogo. Selecciona «Seleccionar imagen local de MicroPython...».



5) Selecciona el firmware de MicroPython «ESP32_GENERIC_C3-20251209-v1.27.0.bin» incluido en el paquete del tutorial. Haz clic en «Abrir».



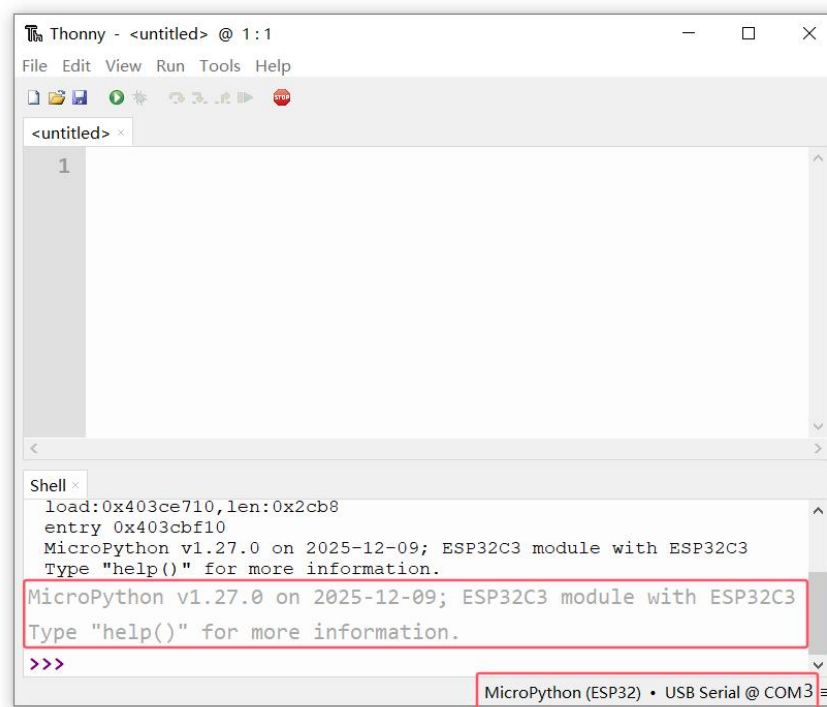
6) Selecciona «USB Serial @ (COM3)» y, a continuación, haz clic en «Instalar».



Espera a que finalice la instalación y, a continuación, haz clic en «Cerrar».

7) Comprobación de la instalación

Una vez finalizada la instalación, puede cerrar la ventana. Debería aparecer el siguiente mensaje en el Shell (véase la imagen de abajo) y, en la esquina inferior derecha, debería aparecer el intérprete que se está utilizando y el puerto COM.



El REPL (Shell)

También deberías ver el **símbolo >>>**

Este símbolo representa el indicador del REPL (Read-Eval-Print-Loop) de MicroPython. Significa que estás interactuando con MicroPython (la placa con el firmware de MicroPython instalado) en modo interactivo y en tiempo real. Esto simplemente significa que puedes introducir y ejecutar comandos directamente.

Escribe `print('Hello')` en el Shell después del indicador `>>>` y pulsa Intro. Debería aparecer «Hello» en el Shell justo después de introducir el comando.

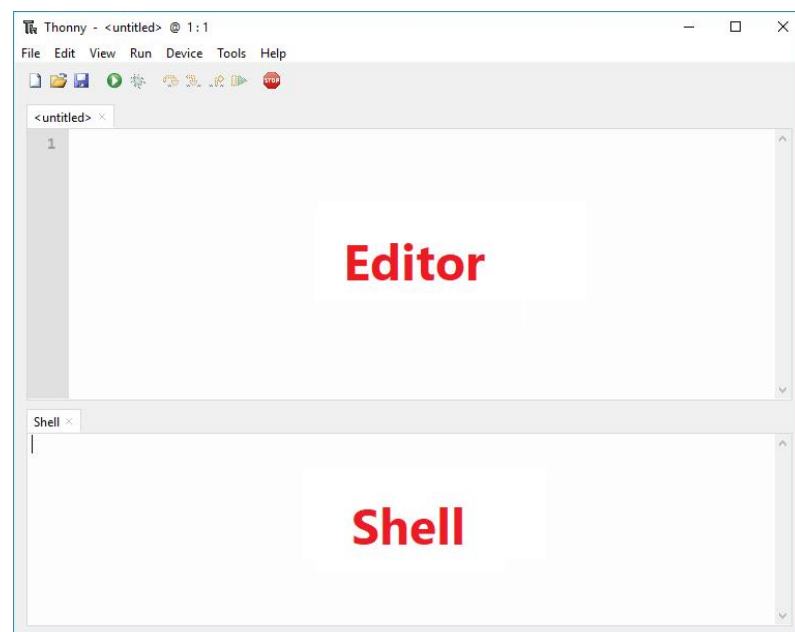
```
>>> print('Hello')
Hello
```

Si todo funciona según lo esperado, significa que has instalado correctamente el IDE Thonny, así como el firmware de MicroPython en tu placa ESP32.

1.5 Descripción general del IDE Thonny

Abre el IDE Thonny. Hay dos secciones diferentes:

el Editor y el Shell/Terminal de MicroPython:



La sección del editor es donde escribes tu código y editas tus archivos .py. Puedes abrir más de un archivo, y el editor abrirá una nueva pestaña para cada uno de ellos.

En el shell de MicroPython, puedes escribir comandos que tu placa ESP32 ejecutará inmediatamente sin necesidad de cargar nuevos archivos. El terminal también

proporciona información sobre el estado de un programa en ejecución, muestra errores relacionados con el proceso de carga, errores de sintaxis, imprime mensajes, etc.

Los iconos

En la parte superior del IDE de Thonny hay varias barras de menú que se utilizan con frecuencia durante la programación:



: Crear un nuevo archivo de Python.



: El icono de la carpeta abierta te permite abrir un archivo existente de tu ordenador. Esto puede resultar útil si vuelves a trabajar en un programa en el que ya habías trabajado anteriormente.



: El icono del disquete te permite guardar tu código. Para evitar la pérdida del código que estás escribiendo, se recomienda adquirir el hábito de hacer clic en este menú con frecuencia.



: Ejecuta el código de Python de la página actual (la que se está viendo) del **editor**.



: Detiene la función que se está depurando.

Las siguientes funciones no son compatibles actualmente con el firmware de MicroPython para ESP32 C3 y se pueden explicar brevemente:



: El icono del insecto te permite depurar tu código.



: La flecha indica a Python que dé un gran paso, es decir, que salte a la siguiente línea o bloque de código.



: La flecha indica a Python que dé un paso pequeño, es decir, que profundice en cada componente de una expresión.



: La flecha indica a Python que salga del depurador.



: Salir del modo de depuración.

1.6 Ejecutar tu primer código: ejecuta el código actual en Thonny

1. Asegúrate de que has configurado el entorno de desarrollo.

En este momento, deberías tener:

- ✓ Una batería 18650 suficientemente cargada instalada y el interruptor de encendido activado.
- ✓ Conectado la placa ESP32 al ordenador.
- ✓ El controlador Ch340 instalado en tu ordenador
- ✓ El IDE Thonny instalado en el ordenador
- ✓ La placa ESP32 programada con el firmware MicroPython

Para que te familiarices con el proceso de escribir un archivo y ejecutar código en tu placa ESP32, vamos a cargar un nuevo script que simplemente controla el zumbador integrado de la ESP32 para producir sonido.

2. Ejecutar el código

Cuando abras el IDE Thonny por primera vez, el editor mostrará un archivo sin título. Copia el siguiente script en ese archivo:

```
from machine import Pin, PWM
import time

buzzer = PWM(Pin(9)) # Inicializa el zumbador en el pin GPIO 9 como salida PWM
while True:
    buzzer.freq(1000) # Establece la frecuencia en 1000 Hz (tono de 1 kHz)
    buzzer.duty(512) # Establece el ciclo de trabajo al 50 % (512/1024) para
    producir sonido
    time.sleep_ms(200) # Mantener el zumbador activo durante 200 milisegundos
    buzzer.duty(0) # Apagar el zumbador estableciendo el ciclo de trabajo al
    0 %
    time.sleep(1) # Espera 1 segundo antes de repetir el pitido
```

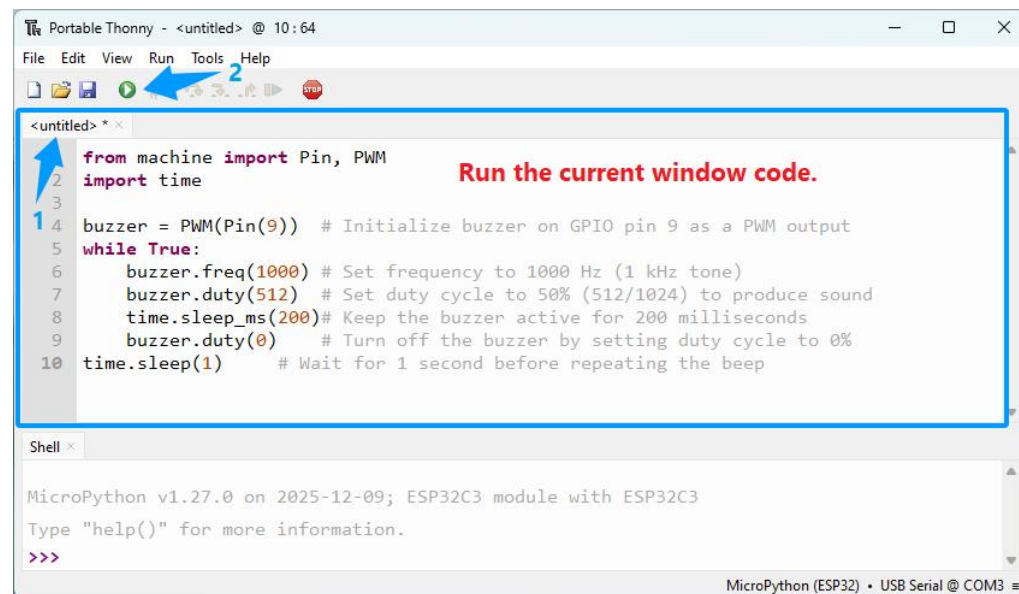
IMPORTANT: How to keep the correct format

```
<untitled> * x
1 from machine import Pin, PWM
2 import time
3
4 buzzer = PWM(Pin(9)) # Initialize buzzer on GPIO pin 9 as a PWM output
5 while True:
6     buzzer.freq(1000) # Set frequency to 1000 Hz (1 kHz tone)
7     buzzer.duty(512) # Set duty cycle to 50% (512/1024) to produce sound
8     time.sleep_ms(200) # Keep the buzzer active for 200 milliseconds
9     buzzer.duty(0) # Turn off the buzzer by setting duty cycle to 0%
10    time.sleep(1) # Wait for 1 second before repeating the beep
```

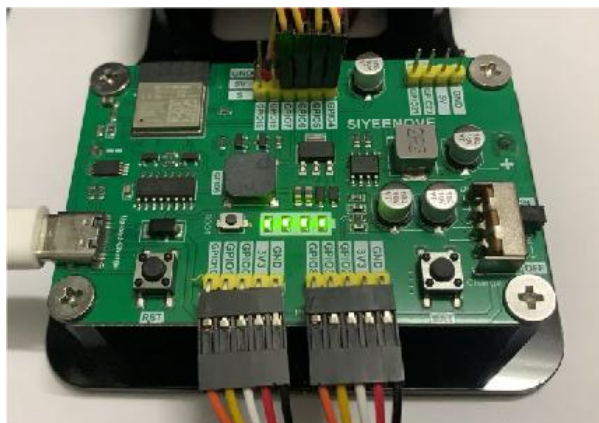
A blank line

Always use 4 spaces for indentation in Python

Al ejecutar el código por primera vez: cuando veas el indicador «>>>», simplemente haz clic en el botón verde «Run» del IDE de Thonny.



Asegúrate de encender el interruptor de la placa de control; el zumbador emitirá entonces un sonido.



Para detener la ejecución del programa, puedes pulsar el botón «STOP» o simplemente pulsar CTRL+C.

Nota:

El mero hecho de ejecutar el archivo con Thonny no lo copia de forma permanente al sistema de archivos del ESP32. Esto significa que, si lo desconectas del ordenador y alimentas la placa, no ocurrirá nada, ya que no tiene ningún archivo de MicroPython guardado en su sistema de archivos.

La función «Ejecutar» del IDE de Thonny es útil para probar el código, pero si quieres subirlo de forma permanente a tu placa, debes crear y guardar un archivo en el sistema de archivos de la placa. Trataremos esto a continuación.

1.7 Carga de código en la placa ESP32: ejecutar código en ESP32

Método 1: Guardar el código como main.py (en el dispositivo ESP32)

1) Después de ejecutar el código del zumbador de la sección anterior (en modo en línea), detén la ejecución haciendo clic en el icono rojo «Stop».

Ahora puedes guardar el código en tu ordenador o subirlo directamente al dispositivo ESP32 con MicroPython:

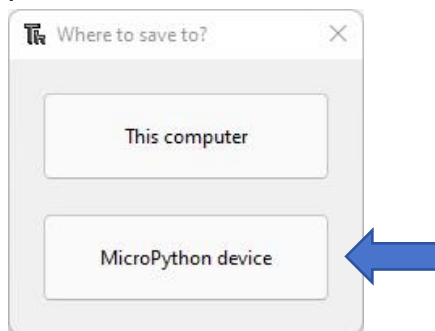
Haz clic en el icono «Guardar» o ve a Archivo > Guardar como...



2) En el cuadro de diálogo de guardado, elige dónde guardar el archivo:

Selecciona «Este ordenador» si quieres guardarlo localmente en tu PC.

Aquí seleccionamos «Dispositivo MicroPython (ESP32)» para subirlo directamente a la placa ESP32.

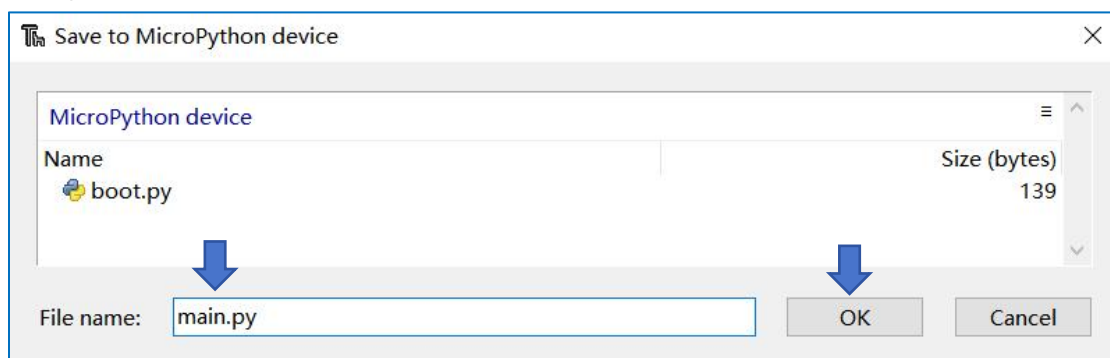


Nota:

P: ¿El panel de archivos de Thonny no muestra «Dispositivo MicroPython»?

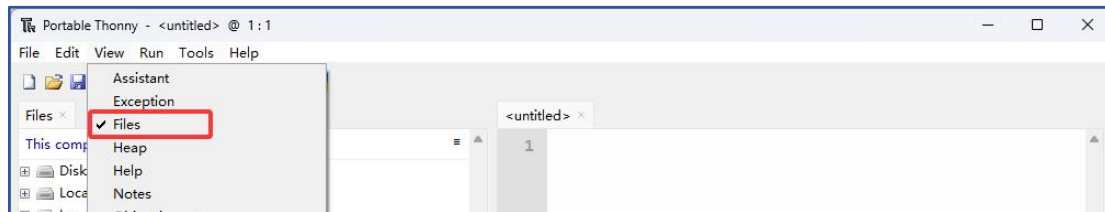
R: Cada vez que se conecta el puerto USB del ordenador a la placa de control, se produce este fenómeno porque el IDE de Thonny no dispone de la función de detectar automáticamente la conexión de dispositivos ESP32. Haz clic en el botón «STOP» o vuelve a seleccionar «MicroPython (ESP32) -USB Serial @ COMx» en la esquina inferior derecha de Thonny para resolver este problema.

3) Asigna al archivo el nombre **main.py** y haz clic en Aceptar. Nota: Guardar el archivo como main.py en el dispositivo ESP32 permite que la placa ejecute este código automáticamente cada vez que se encienda o se reinicie.

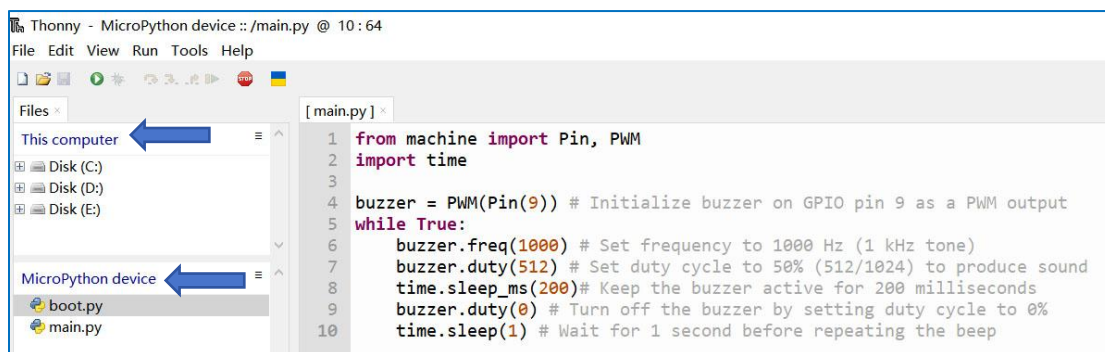


Fíjate en que la ventana que aparece muestra los archivos guardados actualmente en el sistema de archivos de la placa. Debería haber un archivo llamado **boot.py**. Ese archivo se crea de forma predeterminada al grabar el firmware de MicroPython.

4) Haz clic en «Ver» en la barra de menú y selecciona «Archivos» en el menú desplegable.



5) Verás dos paneles de archivos: el panel etiquetado como «Este ordenador» (tus archivos locales) y el panel etiquetado como «Dispositivo MicroPython» (el almacenamiento interno del ESP32 conectado).



6) Tras guardar el archivo, puedes desconectar y volver a conectar la alimentación a la placa. Fíjate en que el zumbador de la placa ESP32 emitirá automáticamente un sonido al conectarle la alimentación. Esto significa que el archivo **main.py** se ha cargado correctamente en el **dispositivo MicroPython** y que este está ejecutando dicho archivo de forma automática.

Importante: cuando nombras un archivo «main.py», el ESP32 lo ejecutará automáticamente al arrancar (al iniciarse). Si le das otro nombre, se guardará igualmente en el sistema de archivos de la placa, pero no se ejecutará automáticamente al arrancar.

¿Se puede ignorar el archivo boot.py?

Sí, puedes ignorar el archivo **boot.py** sin ningún problema. Pero no es necesario borrarlo. En casi todos los proyectos de MicroPython para ESP32 de nivel principiante y en la mayoría de los de nivel intermedio (por ejemplo, parpadeo de un LED, lectura de datos de sensores, conexión WiFi básica, comunicación serie sencilla), el entorno de ejecución de MicroPython del ESP32 funciona con total normalidad, y toda la lógica de tu proyecto se puede escribir directamente en **main.py** (incluidos pasos de inicialización sencillos como la configuración de pines o sensores). Solo es necesario

utilizar boot.py cuando se quiera separar la inicialización a nivel del sistema de la lógica principal de la aplicación para lograr una organización más clara del código. En resumen: boot.py prepara el «entorno del sistema», main.py ejecuta el «proyecto propiamente dicho» y REPL es la «herramienta de depuración»; los tres funcionan conjuntamente para formar la lógica de ejecución completa de MicroPython en ESP32, siendo boot.py un complemento opcional pero potente para la configuración a nivel del sistema.

Método 2: main.py llama a otros archivos

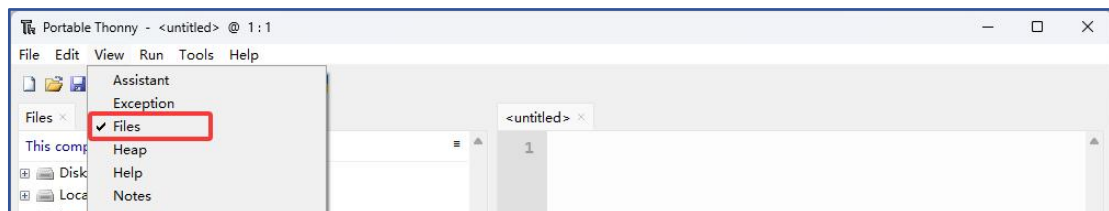
Comparación de los dos métodos

Método	Proceso de funcionamiento	Ventajas	Desventajas	Escenarios aplicables
① Guárdalo como main.py (subida directa)	1. Abre el código del tutorial	Todo el código está escrito en el archivo	1. Alto acoplamiento, lo que dificulta la depuración	Experimentos de un solo archivo, pruebas puntuales
	2. Haz clic en « Guardar como » → Selecciona el dispositivo	main.py, lo cual resulta adecuado para proyectos sencillos de un solo archivo.	2. Requiere volver a cargar todo el código para realizar modificaciones	
	3. Nómbralo main.py		3. No es modular	
② main.py llama a otros archivos	1. Sube el código funcional (p. ej., buzzer.py) al dispositivo	1. Modular, fácil de mantener	Requiere unos pocos pasos más de gestión de archivos	Proyectos con múltiples módulos, desarrollo a largo plazo
	2. Crea un nuevo archivo main.py y escribe el código para llamar a otros archivos desde main.py.	2. Permite la depuración independiente de los submódulos		
	3. Sube main.py al dispositivo ESP32.	3. Admite la reutilización de código		

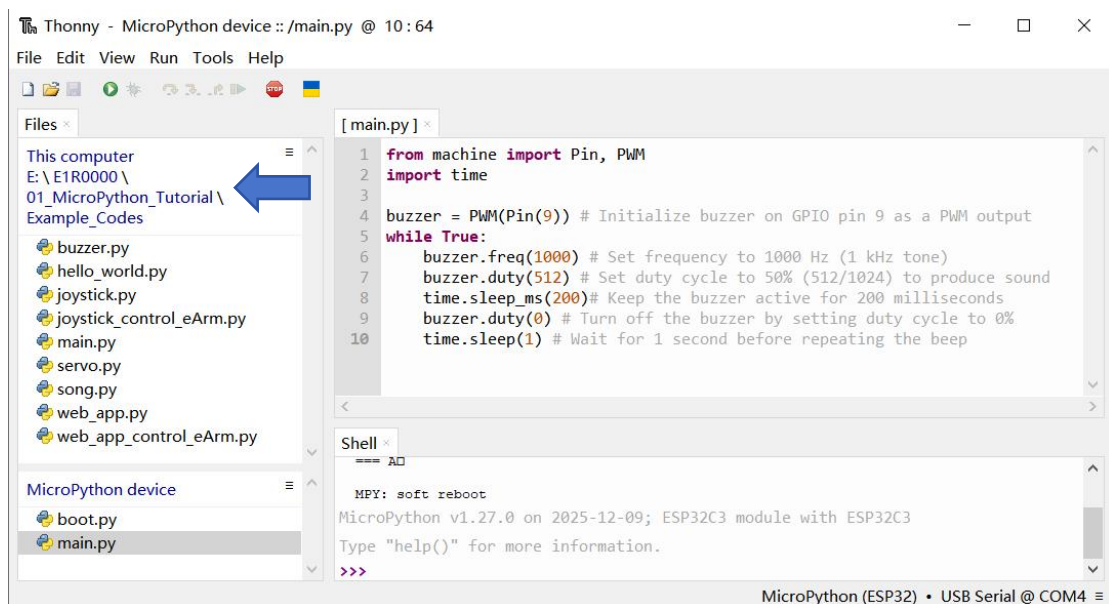
Se recomienda elegir el **segundo** método, ya que es más profesional, flexible y se ajusta a las prácticas recomendadas en el desarrollo embebido.

En el paquete del tutorial te proporcionamos los archivos **xxx.py** y **main.py** con diferentes funciones para que los utilices. El código de **main.py** llamará a cualquiera de los archivos **xxx.py** que se encuentren en «**MicroPython device**», excepto a **main.py** y **boot.py**.

1) Haz clic en «Ver» en la barra de menú y selecciona «Archivos» en el menú desplegable.



2) Haz clic en «Este ordenador», navega hasta la siguiente carpeta donde hayas guardado el paquete del tutorial: **E1R0000\01_MicroPython_Tutorial\Example_Codes** para acceder a los ejemplos de código de MicroPython que se incluyen en el tutorial.

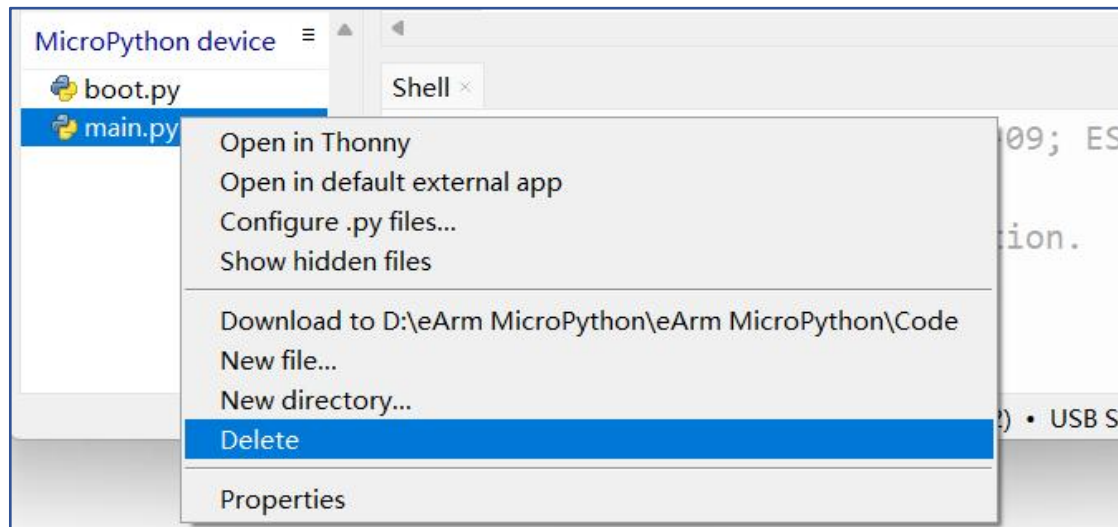


Nota:

P: ¿El panel de archivos de Thonny no muestra «Dispositivo MicroPython»?

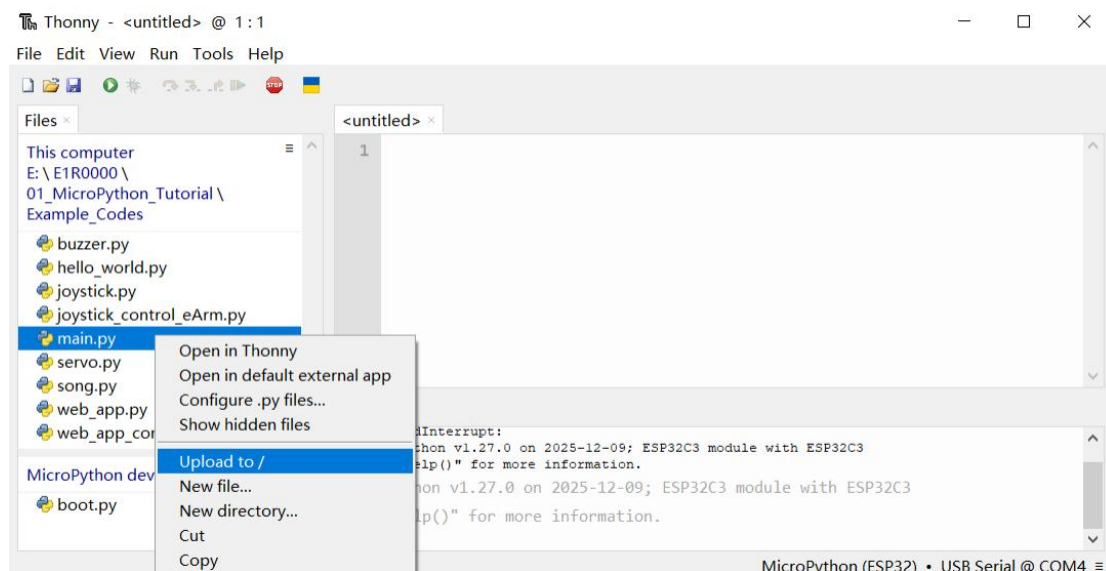
R: Cada vez que se conecta el puerto USB del ordenador a la placa de control, se produce este fenómeno porque Thonny no dispone de la función de detectar automáticamente la conexión de dispositivos ESP32. Haz clic en el botón «**STOP**» o vuelve a seleccionar «**MicroPython (ESP32) -USB Serial @ COMx**» en la esquina inferior derecha de Thonny para resolver este problema.

3) Haz clic con el botón derecho del ratón en «main.py», que habías cargado anteriormente para hacer sonar el zumbador en «Dispositivo MicroPython», y elimínalo del directorio raíz del ESP32-C3.



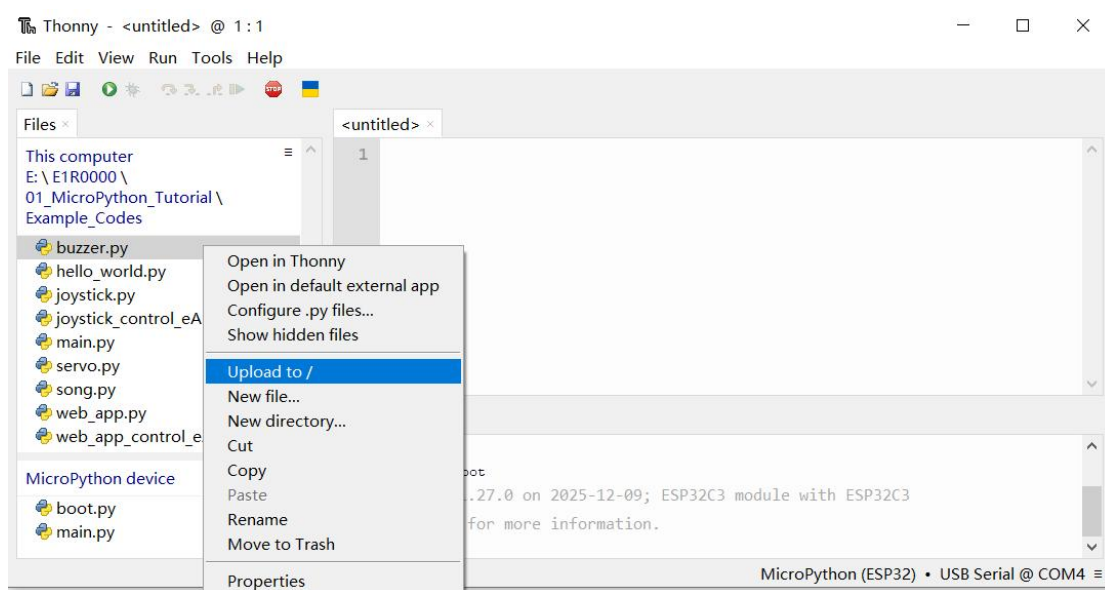
4) Haz clic con el botón derecho del ratón en «main.py» en tu carpeta local («Este ordenador») y selecciona «Subir a/».

Este es el programa principal que te hemos proporcionado. Su finalidad es llamar y ejecutar los demás archivos de ejemplo de MicroPython.

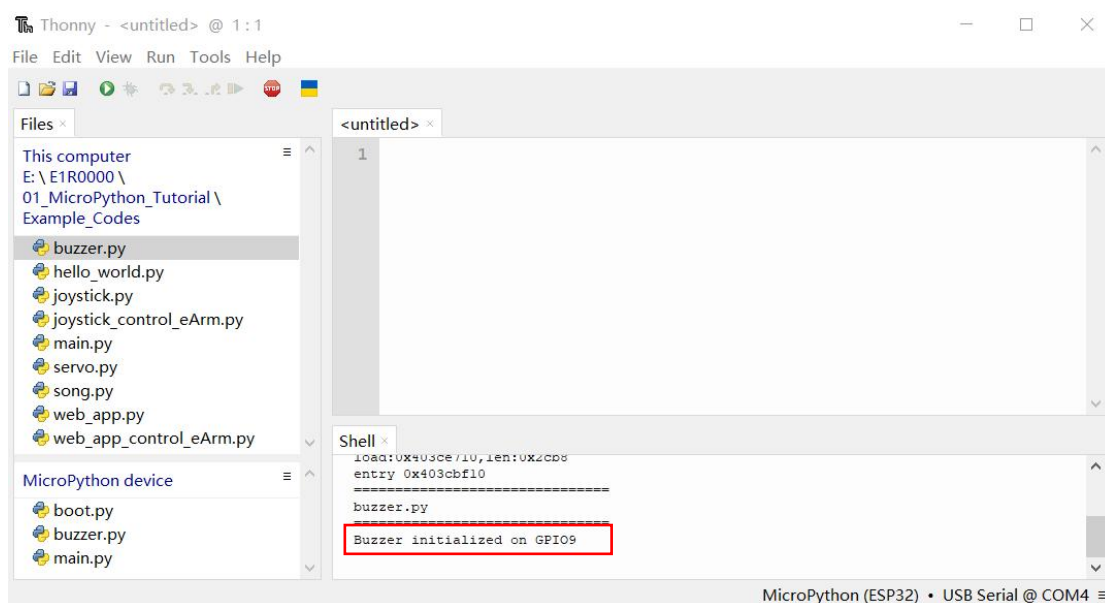


Importante: ; Debes detener la ejecución en línea antes de subir archivos al «dispositivo MicroPython»! (Haz clic en el menú «STOP» en Thonny)

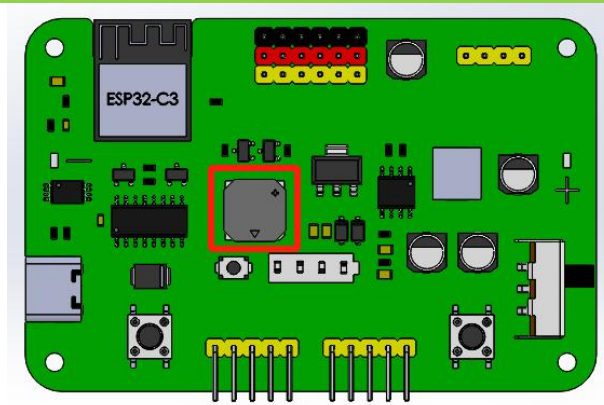
5) Haz clic con el botón derecho del ratón en «buzzer.py» en tu carpeta local y selecciona «Subir a/»



6) Pulsa la tecla de reinicio de la placa ESP32; verás que se ejecuta el código.



7) El zumbador de la placa ESP32-C3 seguirá emitiendo sonidos cuando se active el interruptor de encendido.

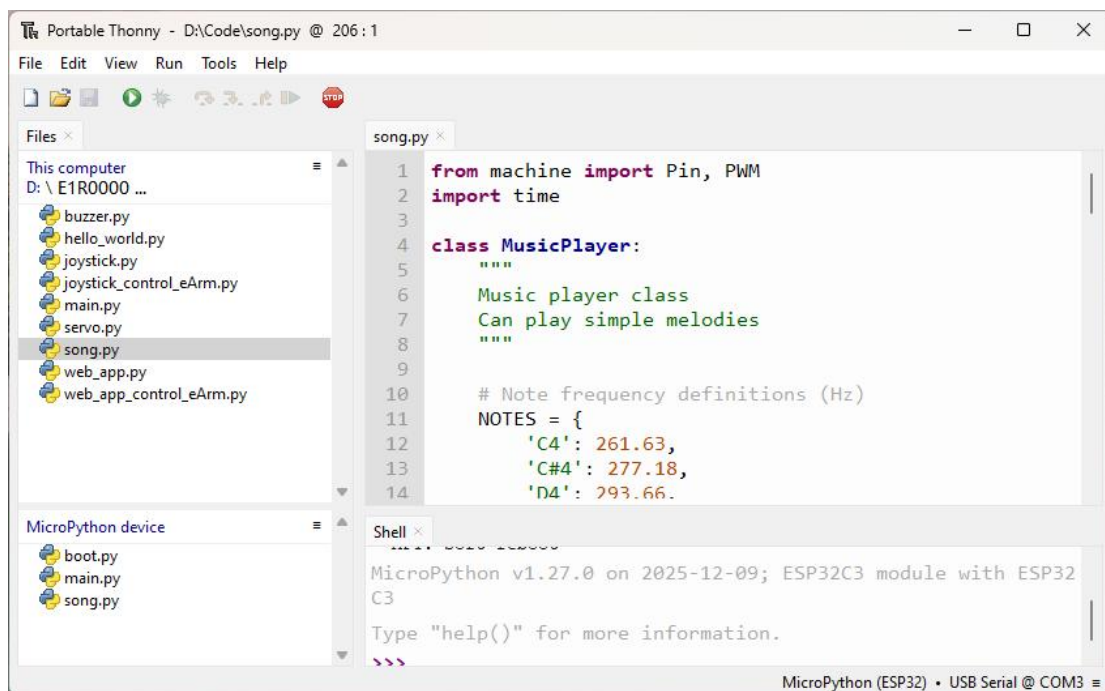


Importante: El archivo «main.py» compartido que te hemos proporcionado está diseñado para llamar solo a uno de los archivos de código que no sean «main.py» cada vez. Si subes varios de estos archivos a la vez, al pulsar el botón de reinicio el sistema podría comportarse de forma errática: podría ejecutar aleatoriamente un archivo diferente en lugar del previsto.

1.8 Ejemplos de código

1.8.1 Reproducir canciones

- 1 Elimina el archivo **buzzer.py** cargado anteriormente del dispositivo MicroPython, ya que el archivo **main.py** proporcionado está diseñado para llamar solo a un archivo de módulo.
- 2 Si hay algún código en ejecución, haz clic en el botón «Detener». A continuación, haz clic con el botón derecho del ratón en **song.py** y selecciona «Subir a /».
- 3 El archivo **song.py** aparecerá en la lista de archivos del dispositivo MicroPython.
- 4 Asegúrate de que el interruptor de encendido de la placa de control esté en la posición ON y de que la batería tenga suficiente carga.
- 5 Pulsa el botón «Reset» de la placa de control esp32 para reiniciarla. A continuación, el zumbador de la placa comenzará a reproducir música.
- 6 Para ver el código: ve al archivo **song.py** y, a continuación, haz doble clic en él para abrirlo en Thonny.



Nota:

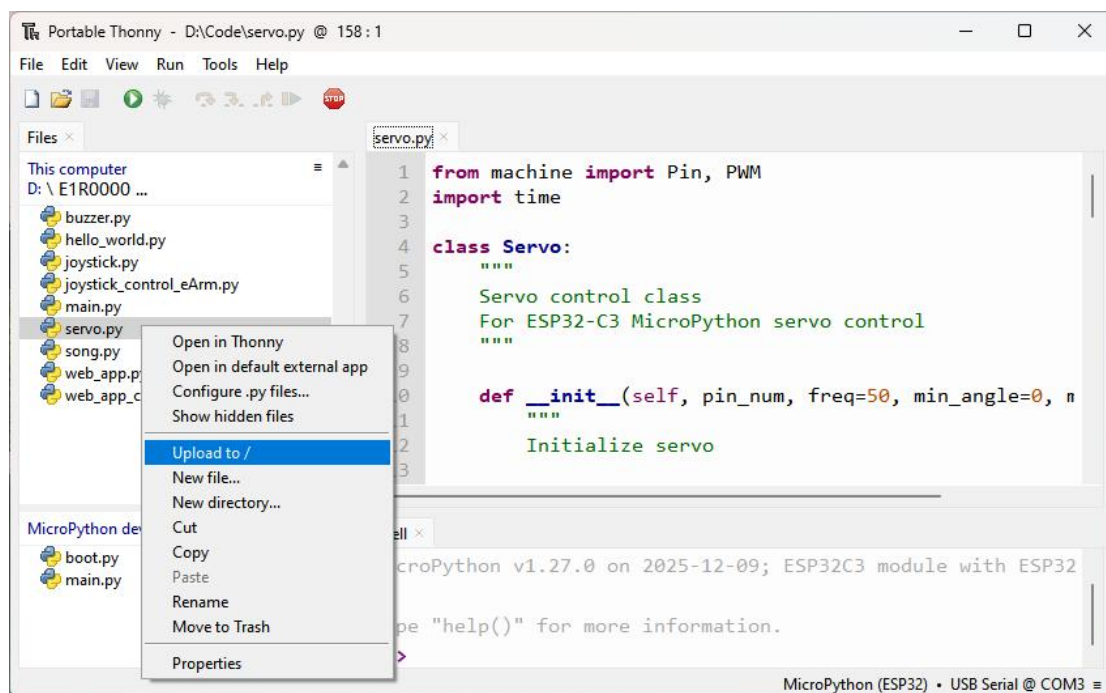
P: ¿El panel de archivos de Thonny no muestra «MicroPython Device»?

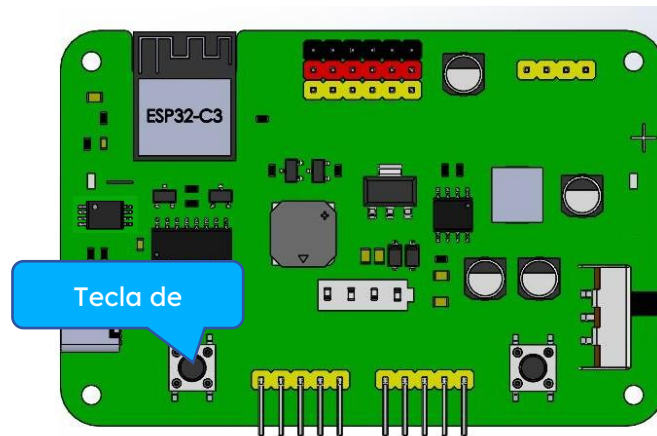
R: Cada vez que se conecta el puerto USB del ordenador a la placa de control, se produce este fenómeno porque Thonny no dispone de la función de detectar automáticamente la conexión de dispositivos ESP32. Haz clic en el botón «STOP» o vuelve a seleccionar «MicroPython (ESP32) •USB Serial @ COMx» en la esquina inferior derecha de Thonny para resolver este problema.



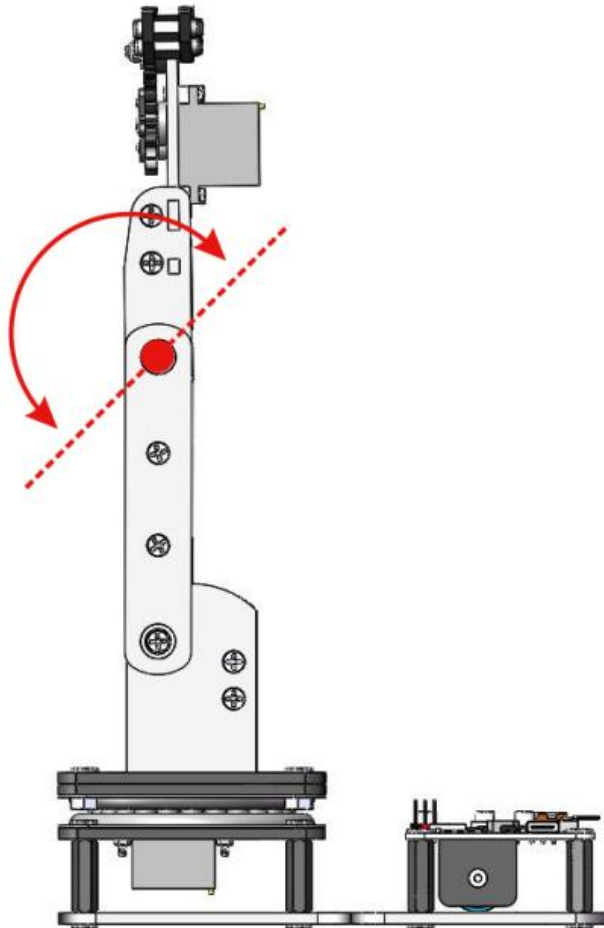
1.8.2 Cómo controlar un servo

- 1 Elimina el archivo **song.py** cargado anteriormente del dispositivo MicroPython, ya que el archivo **main.py** proporcionado está diseñado para llamar solo a un archivo de módulo.
- 2 Si hay algún código en ejecución, haz clic en el botón «Detener». A continuación, haz clic con el botón derecho del ratón en «**servo.py**» y selecciona «Subir a /».
- 3 El archivo «**servo.py**» aparecerá en la lista de archivos del dispositivo de MicroPython.
- 4 Asegúrate de que el interruptor de encendido de la placa de control esté en la posición ON y de que la batería tenga suficiente carga.
- 5 Pulsa el botón de reinicio de la placa de control esp32 para reiniciarla. El servo C del eArm oscilará de 0 a 180 y de 180 a 0 en un ciclo:
- 6 Para **ver** el código: ve al archivo **servo.py** y, a continuación, haz doble clic en él para abrirlo en Thonny.



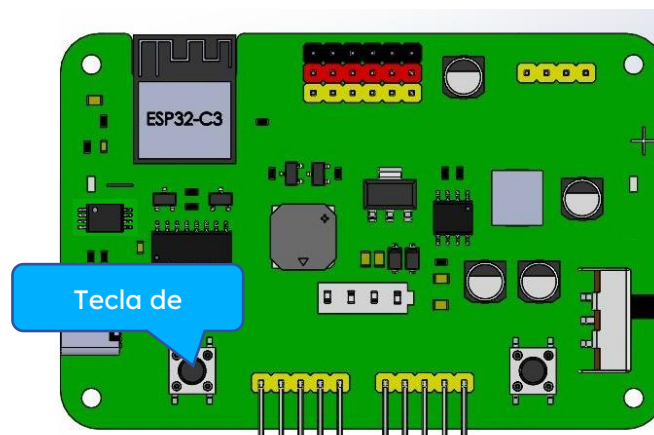
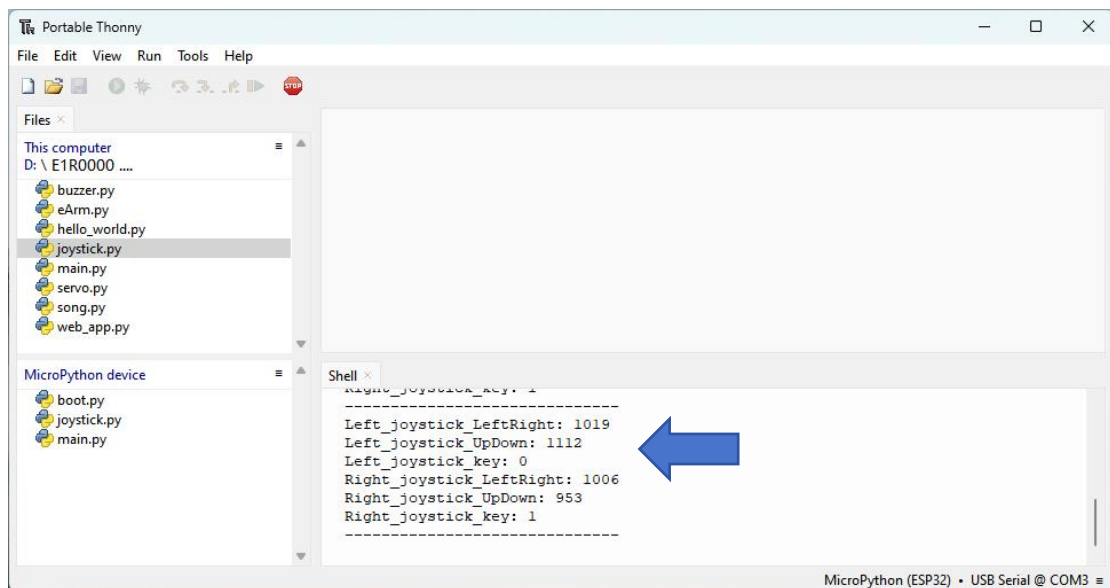


Una vez cargado el código correctamente, el servo C del eArm oscilará de 0 a 180 y de 180 a 0 en un ciclo:



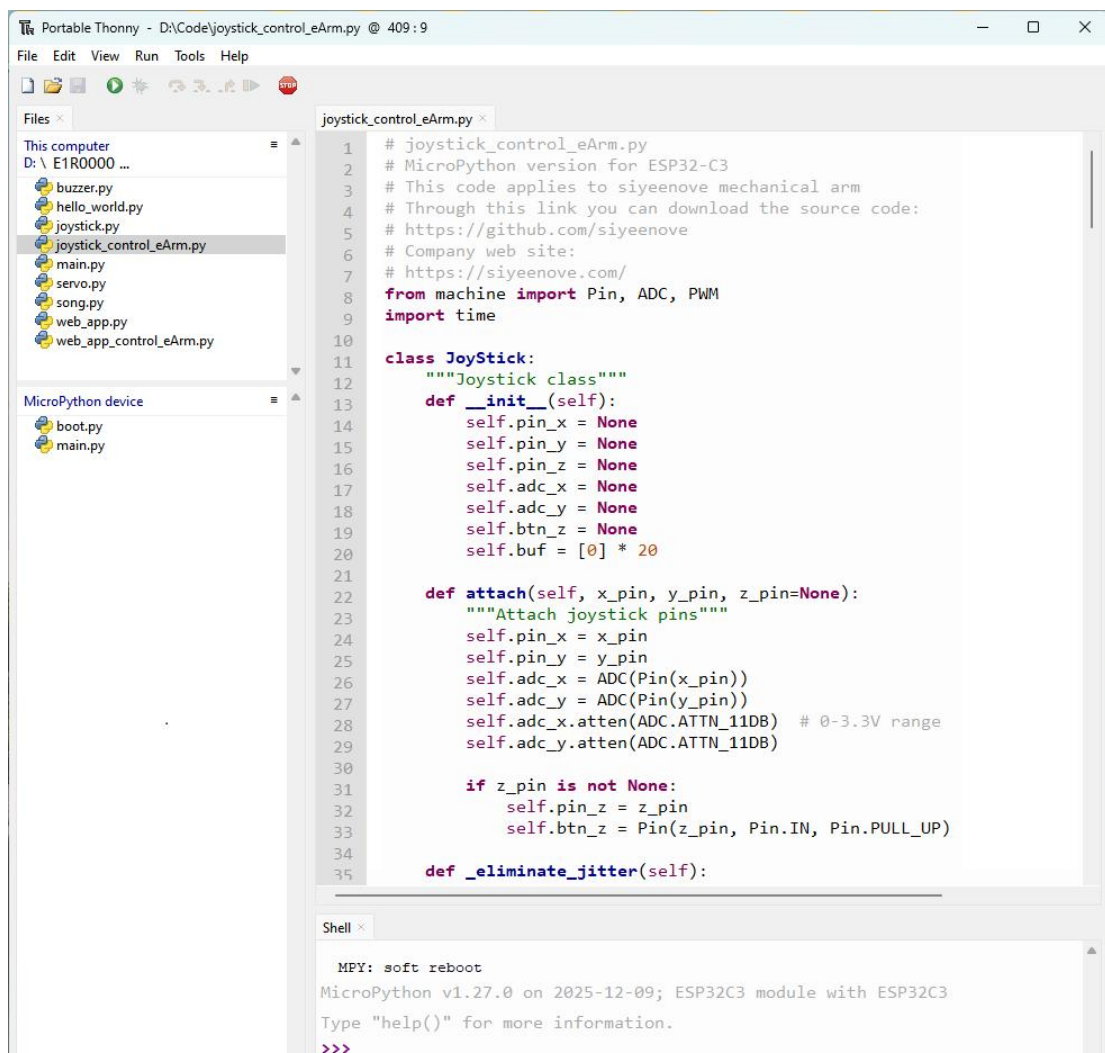
1.8.3 Leer el valor del joystick

- 1 Elimina el archivo **servo.py** cargado anteriormente del dispositivo MicroPython, ya que el archivo **main.py** proporcionado está diseñado para llamar solo a un archivo de módulo.
- 2 Si hay algún código en ejecución, haz clic en el botón «**Detener**». A continuación, haz clic con el botón derecho del ratón en **joystick.py** y selecciona «**Subir a /**».
- 3 El archivo **joystick.py** aparecerá en la lista de archivos del dispositivo MicroPython.
- 4 Asegúrate de que el interruptor de encendido de la placa de control esté en la posición ON y de que la batería tenga suficiente carga.
- 5 Pulsa el botón de **reinicio** de la placa de control esp32 para reiniciarla. Mueve el joystick hacia la izquierda, la derecha, arriba y abajo, y el shell mostrará el valor del joystick
- 6 Para **ver** el código: ve al archivo **joystick.py** y, a continuación, haz doble clic en él para abrirlo en Thonny.



1.8.4 Robot controlado por joystick (con función de grabación y repetición de movimientos)

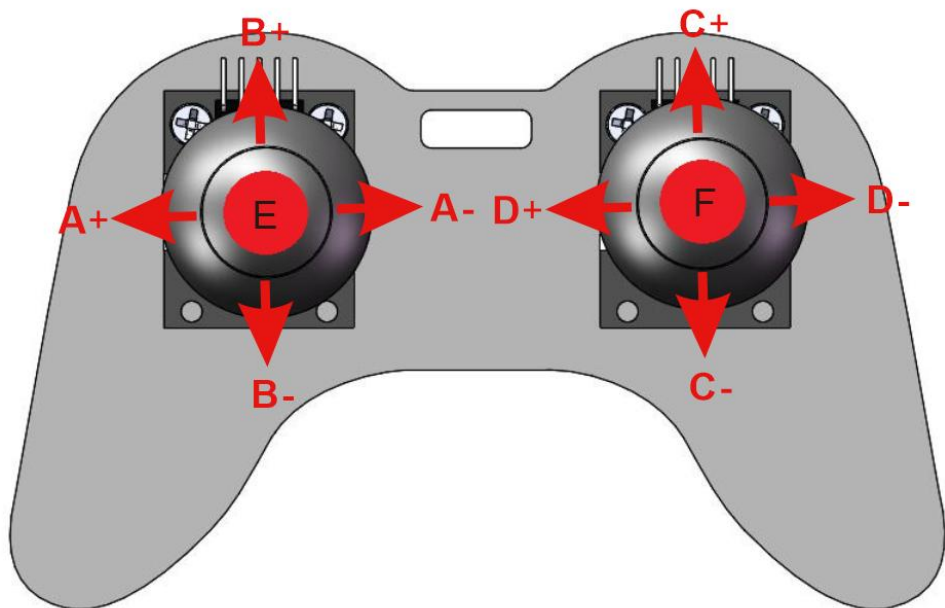
- 1 Elimina el archivo **joystick.py** cargado anteriormente del dispositivo MicroPython, ya que el archivo **main.py** proporcionado está diseñado para llamar solo a un archivo de módulo.
- 2 Si hay algún código en ejecución, haz clic en el botón «Detener». A continuación, haz clic con el botón derecho del ratón en **joystick_control_eArm.py** y selecciona «Subir a /».
- 3 El archivo **joystick_control_eArm.py** aparecerá en la lista de archivos del dispositivo de MicroPython.
- 4 Asegúrate de que el interruptor de encendido de la placa de control esté en la posición ON y de que la batería tenga suficiente carga.
- 5 Pulsa el botón de reinicio de la placa de control esp32 para reiniciarla. Puedes mover o pulsar el joystick para controlar el brazo robótico. Incluso puedes grabar y reproducir los movimientos del robot.
- 6 Para ver el código: ve al archivo **joystick_control_eArm.py** y, a continuación, haz doble clic en él para abrirlo en Thonny.



```
1 # joystick_control_eArm.py
2 # MicroPython version for ESP32-C3
3 # This code applies to siyeenove mechanical arm
4 # Through this link you can download the source code:
5 # https://github.com/siyeenove
6 # Company web site:
7 # https://siyeenove.com/
8 from machine import Pin, ADC, PWM
9 import time
10
11 class JoyStick:
12     """Joystick class"""
13     def __init__(self):
14         self.pin_x = None
15         self.pin_y = None
16         self.pin_z = None
17         self.adc_x = None
18         self.adc_y = None
19         self.btn_z = None
20         self.buf = [0] * 20
21
22     def attach(self, x_pin, y_pin, z_pin=None):
23         """Attach joystick pins"""
24         self.pin_x = x_pin
25         self.pin_y = y_pin
26         self.adc_x = ADC(Pin(x_pin))
27         self.adc_y = ADC(Pin(y_pin))
28         self.adc_x.atten(ADC.ATTN_11DB) # 0-3.3V range
29         self.adc_y.atten(ADC.ATTN_11DB)
30
31         if z_pin is not None:
32             self.pin_z = z_pin
33             self.btn_z = Pin(z_pin, Pin.IN, Pin.PULL_UP)
34
35     def _eliminate_jitter(self):
```

```
MPY: soft reboot
MicroPython v1.27.0 on 2025-12-09; ESP32C3 module with ESP32C3
Type "help()" for more information.
>>>
```

Control mediante joystick – Guía de funcionamiento del brazo robótico:

	
A+: Controla el servomotor A; el brazo robótico gira hacia la izquierda.	A-: Controla el servomotor A; el brazo robótico gira a la derecha.
B+: controla el servomotor B; el brazo grande del brazo robótico se balancea hacia delante.	B-: Controla el servomotor B; el brazo grande del brazo robótico se balancea hacia atrás.
C+: Controla el servomotor C; el brazo pequeño del brazo robótico se balancea hacia delante.	C-: Controla el servomotor C; el brazo pequeño del robot se balancea hacia atrás.
D+: Controla el servomotor D; la pinza del brazo robótico se abre (se afloja).	D-: Controla el servomotor D; la pinza del brazo robótico se cierra (se aprieta).
E: Grabar acción – Pulsa el botón E una vez para grabar una acción (se pueden grabar hasta 20 acciones seguidas). Cada vez que se pulsa, se oye un pitido corto. Cuando se alcanza la capacidad máxima de grabación, el zumbador emite un pitido largo.	F: Reproducción — Tras grabar la secuencia, pulsa una vez el botón F para iniciar la reproducción en bucle de las acciones grabadas de forma continua. El zumbador emitirá un pitido y el brazo repetirá las acciones en bucle. Para salir del bucle, mantén pulsado el botón F (pulsación prolongada); el zumbador emitirá un pitido largo para indicar que la reproducción se ha detenido.

1.8.5 Leer el valor de la aplicación web

- 1 Elimina el archivo **joystick_control_eArm.py** cargado anteriormente del dispositivo MicroPython, ya que el archivo **main.py** proporcionado está diseñado para llamar solo a un archivo de módulo.
- 2 Si hay algún código en ejecución, haz clic en el botón «Detener». A continuación, haz clic con el botón derecho del ratón en **web_app.py** y selecciona «Subir a /»
- 3 El archivo **web_app.py** aparecerá en la lista de archivos del dispositivo MicroPython.
- 4 Asegúrate de que el interruptor de encendido de la placa de control esté en la posición ON y de que la batería tenga suficiente carga.
- 5 Pulsa el botón «Reset» de la placa de control esp32 para reiniciarla. Podrás ver la información de inicio del servicio de red del esp32 en el Shell. Una vez conectado a la red WiFi del ESP32, abre la aplicación web en un navegador. Cada vez que pulses un botón, su valor se registrará en el Shell.
- 6 Para ver el código: ve al archivo **web_app.py** y, a continuación, haz doble clic en él para abrirlo en Thonny.

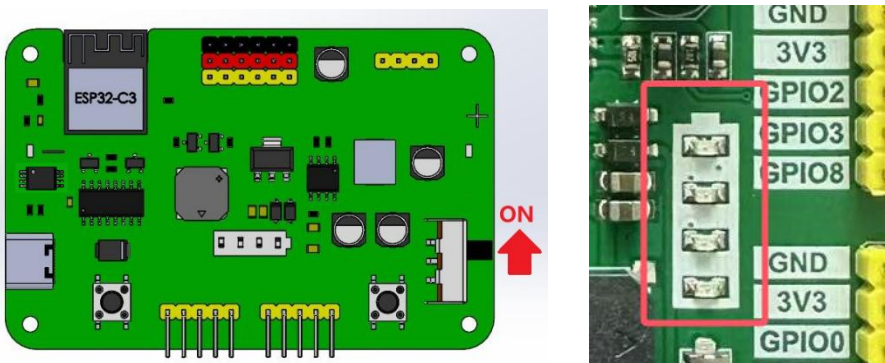
Una vez cargado el código, el Shell mostrará la información de la red wifi.



Para conectarte a la red Wi-Fi, sigue los pasos que se indican a continuación.

1) Coloca el interruptor de encendido del eArm en la posición ON.

- : Asegúrate de que la batería de litio 18650 esté instalada.
- : El indicador de batería muestra 3 barras o más.

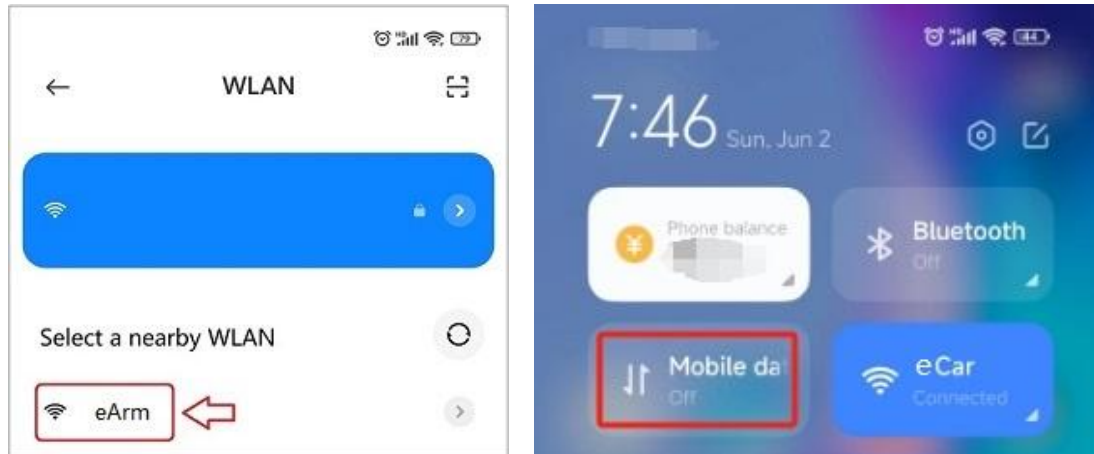


2) Activa el Wi-Fi del teléfono y conéctate a la red denominada «eArm»

■: Una vez conectado al Wi-Fi, es posible que aparezca un mensaje que dice «Error en la conexión de red»; simplemente ignóralo.

■: Desactiva los datos móviles en los ajustes de tu teléfono

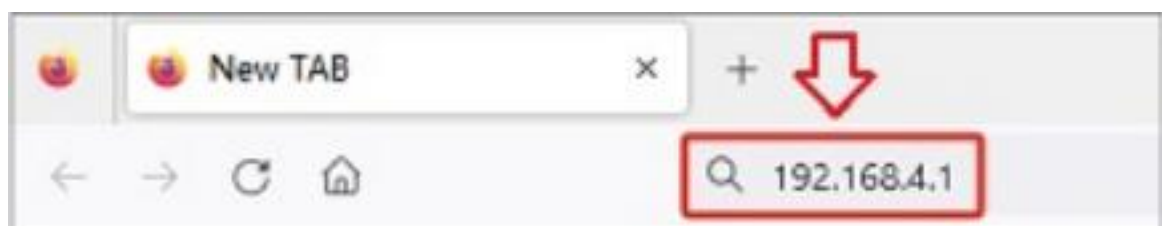
■: Esto garantiza una conexión estable a la aplicación web



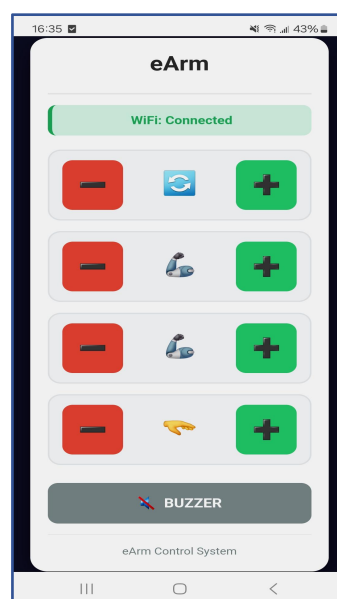
3) Abre el navegador web de tu teléfono

4) Escribe 192.168.4.1 en la barra de direcciones

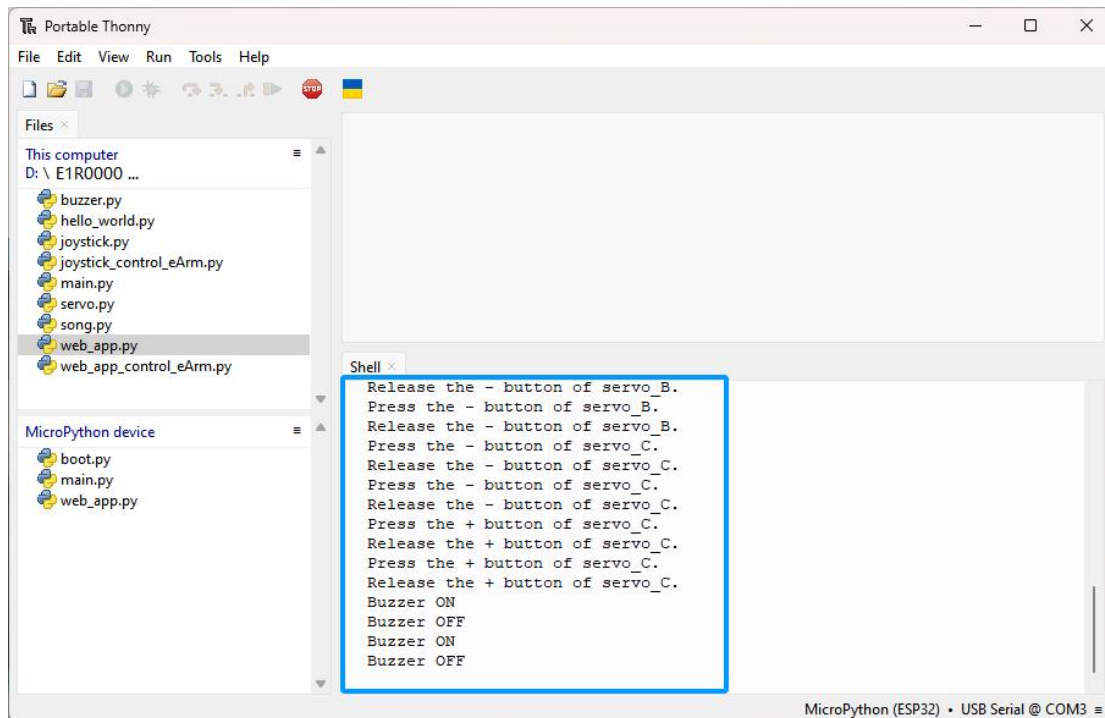
5) Pulsa Intro para conectarte



Una vez establecida la conexión wifi, aparecerá la interfaz de control de la aplicación en tu navegador.



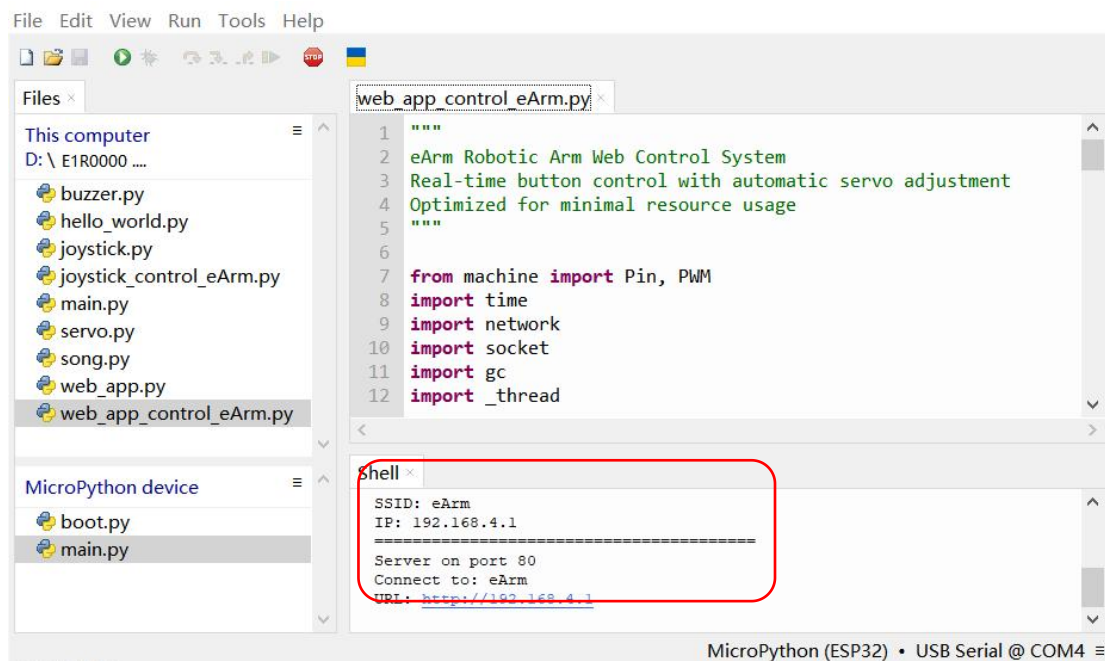
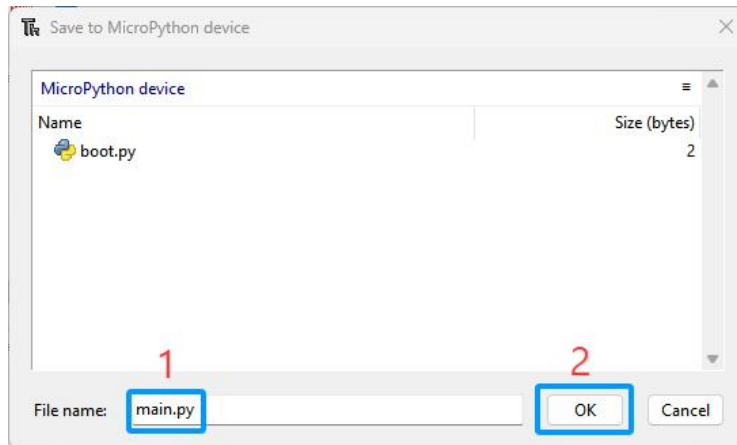
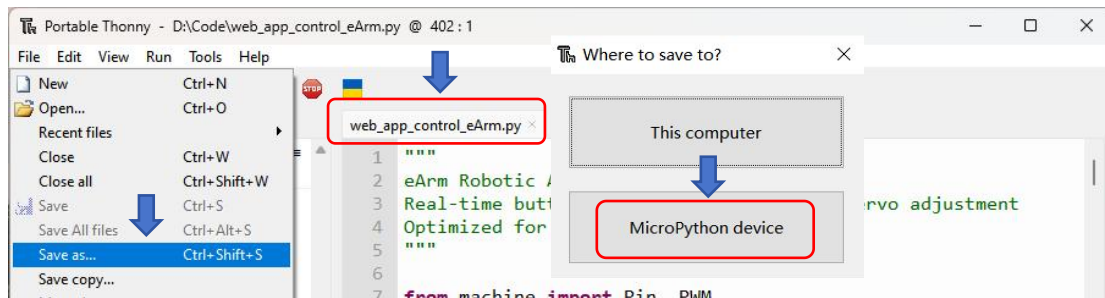
Haz clic en el botón de la aplicación web y el shell mostrará la información clave.



1.8.6 Brazo robótico controlado mediante la aplicación web

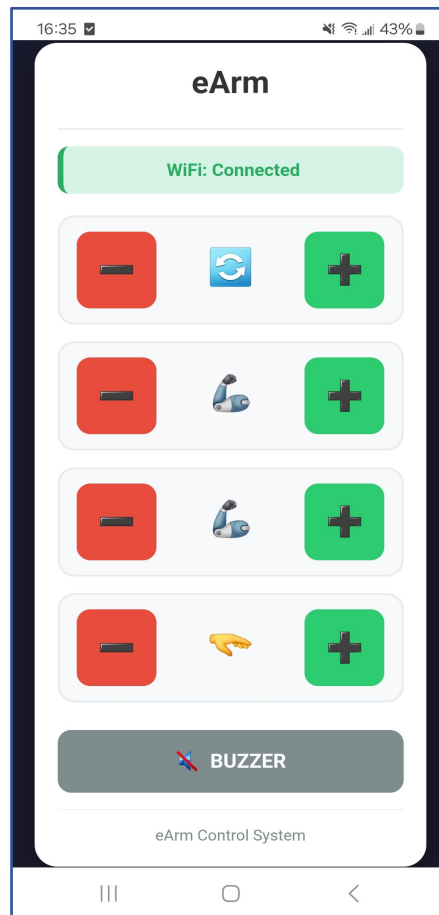
- 1 Elimina los archivos **web_app.py** y **main.py** cargados anteriormente del dispositivo MicroPython.
- En esta última lección, **guarda** el código directamente **como main.py** y cárgalo en el dispositivo MicroPython. Evita la sobrecarga de memoria y asegúrate de que todo funcione correctamente.
- 2 Ve al archivo **web_app_control_eArm.py** y, a continuación, haz doble clic en él para abrirlo en Thonny.
- 3 Haz clic en **Archivo > Guardar como...** y selecciona el dispositivo MicroPython.
- 4 Asigna al archivo el nombre **main.py** y haz clic en Aceptar para continuar.
- 5 Asegúrate de que el interruptor de encendido de la placa de control esté en la posición ON y de que la batería tenga suficiente carga.
- 6 **Pulsa** el botón «Reset» de la placa de control ESP32 para reiniciarla. Podrás ver la información de inicio del servicio de red del ESP32 en el Shell. Una vez conectado a la red Wi-Fi del ESP32, puedes abrir la aplicación web en un navegador para controlar el brazo robótico





Conéctate a la red Wi-Fi del ESP32 siguiendo el método de la lección anterior, o haz clic en [aquí](#).

Una vez que tu dispositivo se haya conectado a la red Wi-Fi del ESP32, puedes abrir la aplicación web en un navegador para controlar el brazo robótico.



		El brazo robótico gira a la izquierda
		El brazo robótico gira a la derecha
		El brazo trasero se eleva
		El brazo trasero baja
		El brazo delantero se eleva
		El brazo delantero baja
		La pinza se abre
		Cierre de la pinza
		Zumbador ENCENDIDO
		Zumbador desactivado

¡ Importante! Mantenimiento crítico de la pinza (protección del servo)

Sujetar continuamente un objeto bajo carga ejerce una tensión considerable sobre el servomotor (especialmente sobre el eje D que acciona la pinza), lo que provoca que genere calor. Por lo tanto, recomendamos encarecidamente limitar la sujeción continua con carga a un máximo de 40 segundos. Tras completar una operación de sujeción, suelta el objeto inmediatamente abriendo la pinza para permitir que el motor se enfríe y descanse. Superar esta duración es la causa principal del sobrecalentamiento y la quemadura del motor.

Nuestro último firmware/código optimizado incluye una función de protección. Si el servo de la pinza no recibe un nuevo comando en un plazo de 40 segundos, se desactivará automáticamente (cesará la transmisión de la señal PWM) para evitar el sobrecalentamiento.

Práctica recomendada: Controla activamente la pinza. Cuando no estés manipulando objetos, abre la pinza o envía comandos para mantenerla en un estado relajado.

2

Flashear el firmware

Proporcionar el «programa de restablecimiento de fábrica» del dispositivo directamente a los usuarios en forma de archivo de firmware (por ejemplo, .bin o .hex), lo que les permite flashearlo en el dispositivo sin necesidad de configurar un IDE Thonny, completando así el proceso de restauración.

1. Problemas con el enfoque tradicional

Normalmente, restaurar un dispositivo mediante Thonny requiere que los usuarios:

- Instalar el IDE de Thonny o las herramientas de desarrollo relacionadas.
- Descargar el código fuente (archivo .py), lo que puede requerir la configuración de bibliotecas adicionales y dependencias.
- Compilarlo y subirlo al dispositivo.

Este proceso supone una gran barrera para los usuarios sin conocimientos técnicos y puede fallar debido a problemas relacionados con el entorno.

2. Ventajas de proporcionar el firmware directamente

Proceso simplificado para el usuario:

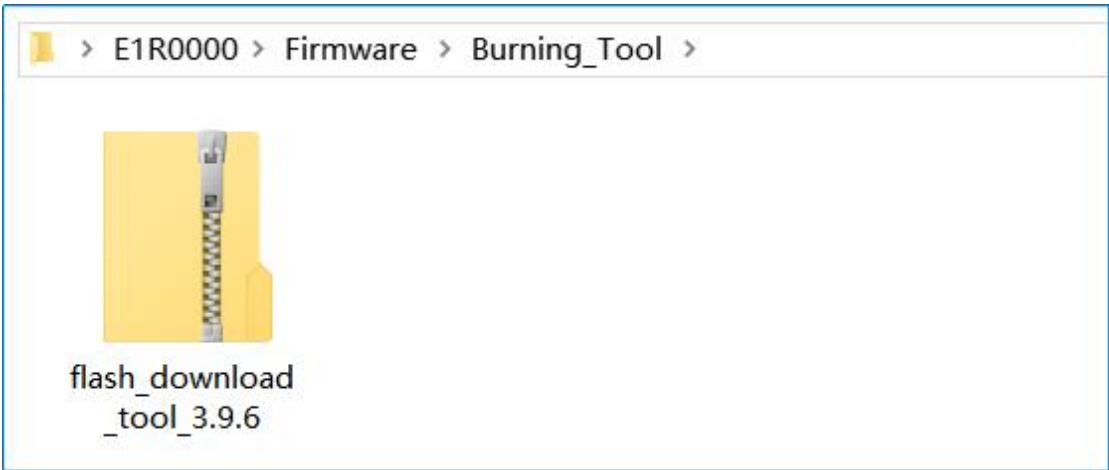
- ▶ Proporciona un archivo de firmware precompilado (por ejemplo, firmware.bin), lo que permite a los usuarios flashearlos con una herramienta sencilla (por ejemplo, un flasheador serie) en un solo paso.
- ▶ No es necesario instalar el IDE Thonny, resolver errores de compilación ni gestionar conflictos entre bibliotecas.

Casos de uso:

- ▶ Restablecer la configuración de fábrica de un dispositivo que no funciona correctamente.
- ▶ Actualización masiva del mismo firmware en varios dispositivos (mejora la eficiencia).

2.1 Herramienta de grabación «flash_download_tool»

La herramienta de grabación se encuentra en la siguiente carpeta:



También puedes descargar la última versión de la herramienta de flasheo desde la página web oficial:

<https://www.espressif.com/en/support/download/other-tools>

Flash Download Tools

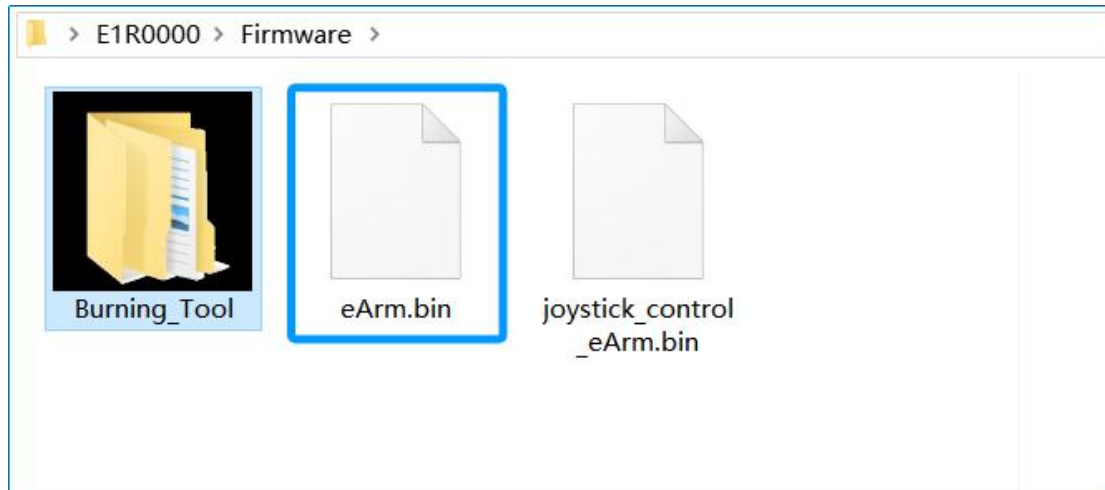
Expand all +

Please refer to Flash Download Tool User Guide to download this tool.

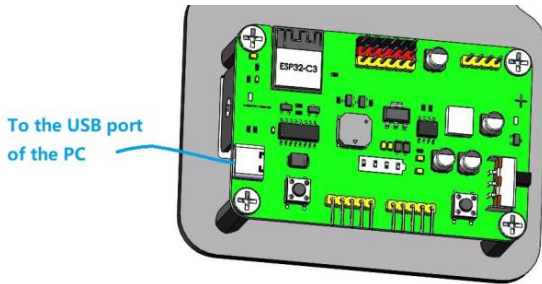
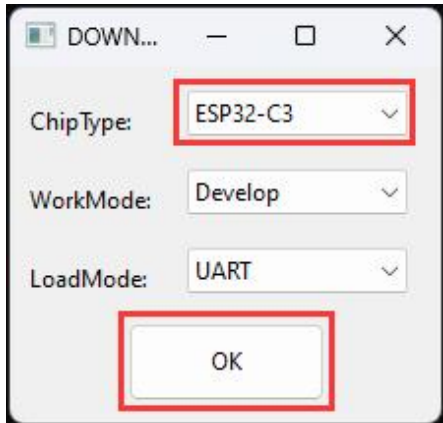
Title	Platform	Version	Release Date ▾	Download
+ Flash Download Tools	HTML	latest	2024.12.18	

2.2 Firmware eArm de (firmware de fábrica)

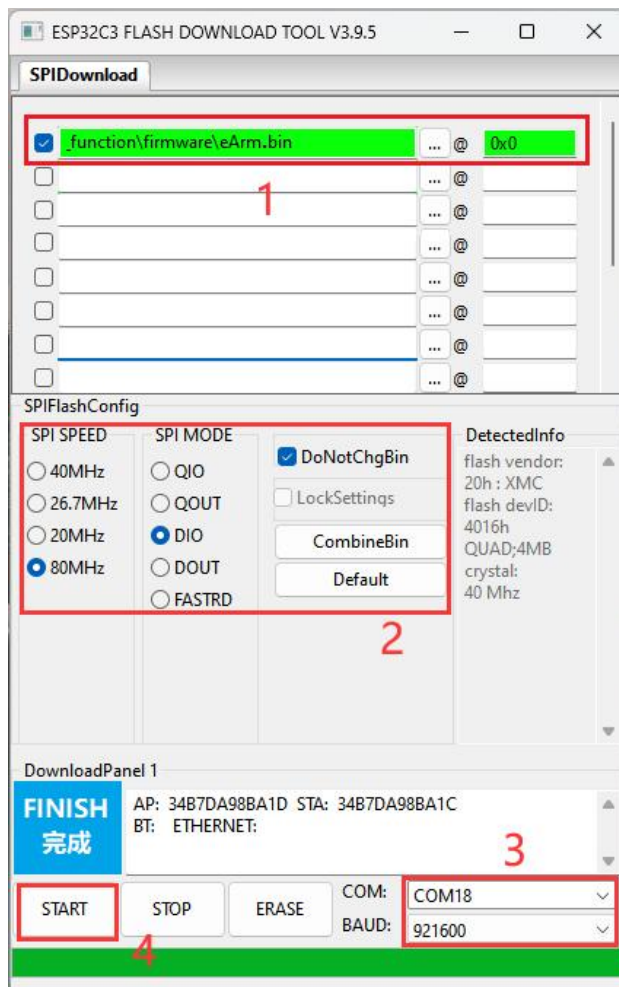
Este firmware tiene exactamente la misma funcionalidad que el brazo robótico de fábrica. Cuando se pierden las funciones de fábrica, este firmware se puede utilizar para restaurarlas. Se encuentra guardado en el siguiente archivo:



2.2.1 Flashear el firmware del eArm con

Utiliza el cable USB para conectar el eArm al ordenador:	Inicia la herramienta de grabación:
	

Selecciona el firmware que deseas grabar e introduce correctamente la dirección de grabación.



Debe estar instalado el controlador CH340, la batería debe estar correctamente conectada y el interruptor de encendido del eArm debe estar activado; de lo contrario, no se detectará el puerto COM.

1: Marca una de las opciones de grabación «», a continuación haz clic en

«» y selecciona el archivo de firmware «**eArm.bin**» que se proporciona en nuestro tutorial. A continuación, introduce correctamente la dirección de grabación (**0x0**) después de «@»:

Nombre del archivo	Dirección de grabación
eArm.bin	0x0

2: Selecciona **80 MHz** en «SPI SPEED», «**DIO**» en «SPI MODE» y marca la casilla

«» en «**DotNotChgBin**».

3: Selecciona el puerto serie de la placa de control. En este caso, es el COM18 (selecciona el puerto según el que tu ordenador haya asignado realmente a este dispositivo). Configura la velocidad de transmisión en **921 600**. Si la grabación falla, puedes reducir la velocidad de transmisión, por ejemplo, a **115 200**.

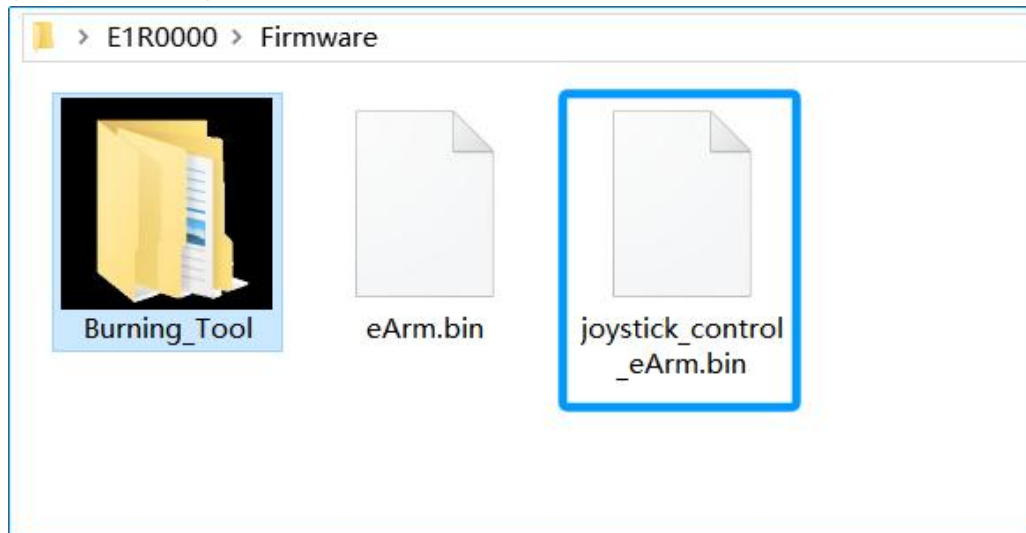
4: Haz clic para iniciar la grabación del firmware.

Una vez completada la grabación, ¡puedes seguir la «**Guía de montaje y uso**» para controlar el brazo robótico!

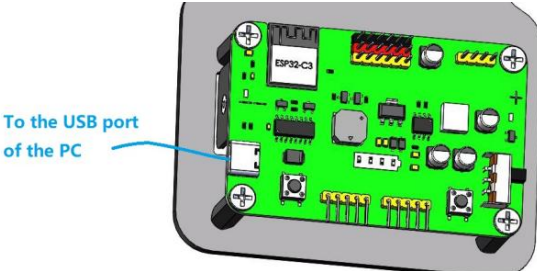
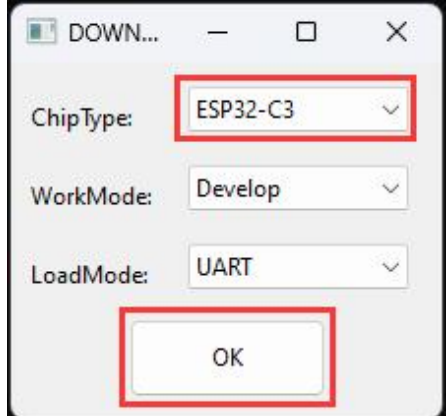
Nota: Una vez finalizada la actualización del firmware, debes pulsar el botón RST (Reset) o desconectar el cable USB y, a continuación, encender el dispositivo.

2.3 Firmware de control mediante joystick « » (con función de grabación)

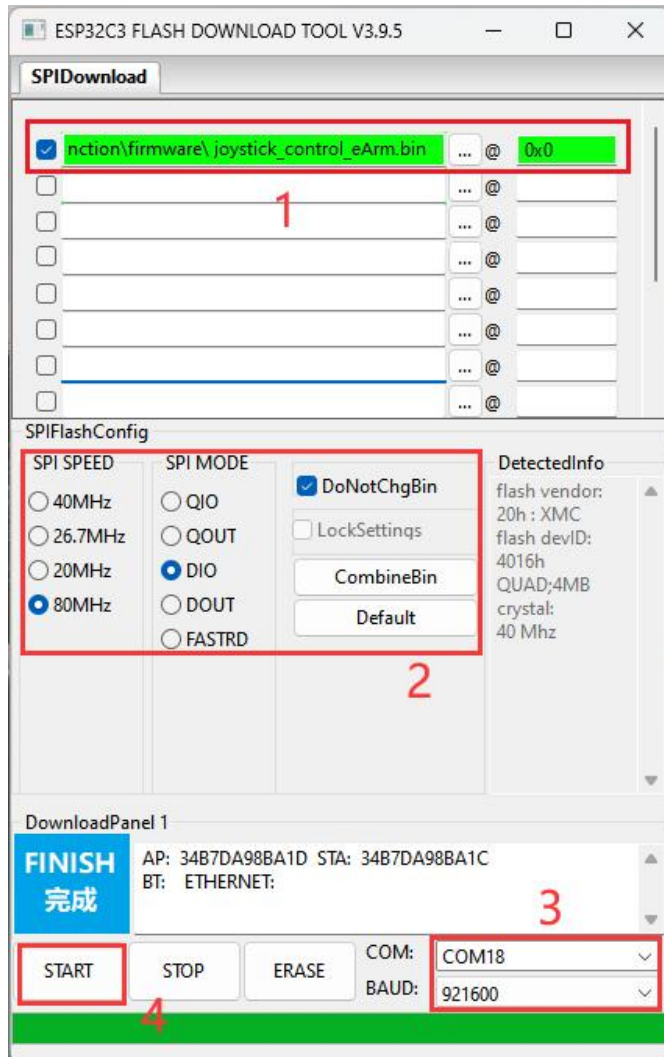
Este firmware tiene las mismas funciones que el código de la sección 4.8.4, con una capacidad de grabación de 20 acciones. Una vez actualizado este firmware, las funciones de control mediante joystick y de grabación y reproducción de movimientos estarán disponibles sin necesidad de instalar Thonny. El firmware se guarda en el siguiente archivo:



2.3.1 Grabar el firmware de control del joystick de

Utiliza el cable USB para conectar el eArm al ordenador:	Inicia la herramienta de grabación:
	

Selecciona el firmware que deseas grabar e introduce correctamente la dirección de grabación.



El controlador CH340 debe estar instalado, la batería debe estar correctamente conectada y el interruptor de encendido del eArm debe estar activado; de lo contrario, no se

1: Marca una de las opciones de grabación «», a continuación haz clic en «» y selecciona el archivo de firmware «[joystick_control_eArm.bin](#)» que se proporciona en nuestro tutorial. A continuación, introduce correctamente la dirección de grabación (0x0) después de «@»:

Nombre del archivo	Dirección de grabación
joystick_control_eArm.bin	0x0

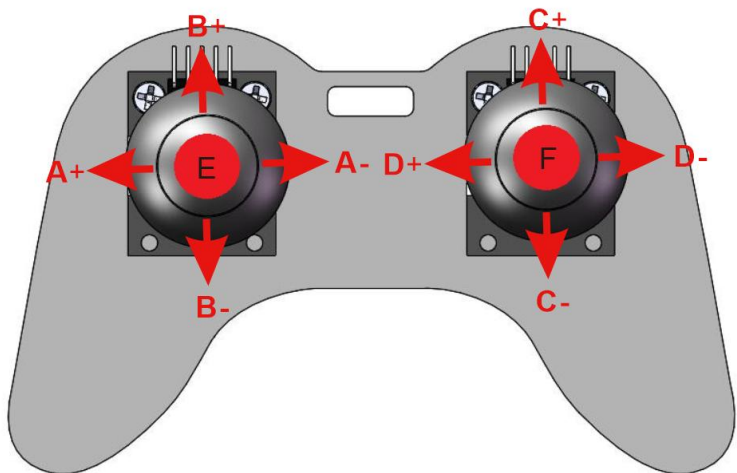
2: Selecciona **80 MHz** para «SPI SPEED», «**DIO**» para «SPI MODE» y marca la casilla «» en «**DotNotChgBin**».

3: Selecciona el puerto serie para la placa de control. En este caso, es el COM18 (selecciona el puerto según el que tu ordenador haya asignado realmente a este dispositivo). Establece la velocidad de transmisión en **921 600**. Si la grabación falla, puedes reducir la velocidad de transmisión, por ejemplo, a **115 200**.

4: Haz clic para iniciar la grabación del firmware.

Una vez completada la actualización del firmware, debes pulsar el botón RST (Reset) o desconectar el cable USB y, a continuación, encender el dispositivo.

Control mediante joystick - Guía de funcionamiento del brazo robótico:

	
A+: Controla el servo A; el brazo robótico gira hacia la izquierda.	A-: Controla el servo A; el brazo robótico gira a la derecha.
B+: controla el Bservo; el brazo grande del brazo robótico se balancea hacia delante.	B-: controla el servomotor B; el brazo grande del brazo robótico se balancea hacia atrás.
C+: Controla el servomotor C; el brazo pequeño del brazo robótico se balancea hacia delante.	C-: Controla el servomotor C; el brazo pequeño del robot se balancea hacia atrás.
D+: Controla el servomotor D; la pinza del brazo robótico se abre (se afloja).	D-: Controla el servo D; la pinza del brazo robótico se cierra (se aprieta).
E: Grabar acción - Pulsa el botón E una vez para grabar una acción (se pueden grabar hasta 20 acciones seguidas). Cada vez que se pulsa, se oye un pitido corto. Cuando se alcanza la capacidad máxima de grabación, el zumbador emite un pitido largo.	F: Reproducción - Una vez grabada la secuencia, pulsa una vez el botón F para iniciar la reproducción en bucle de las acciones grabadas de forma continua. El zumbador emitirá un pitido y el brazo repetirá las acciones en bucle. Para salir del bucle, mantén pulsado el botón F (pulsación prolongada); el zumbador emitirá un pitido largo para indicar que la reproducción se ha detenido.

3

Problemas habituales y soluciones

Problemas comunes y soluciones para la programación del ESP32-C3 con el IDE Thonny (versión para Windows) — Preguntas frecuentes para principiantes

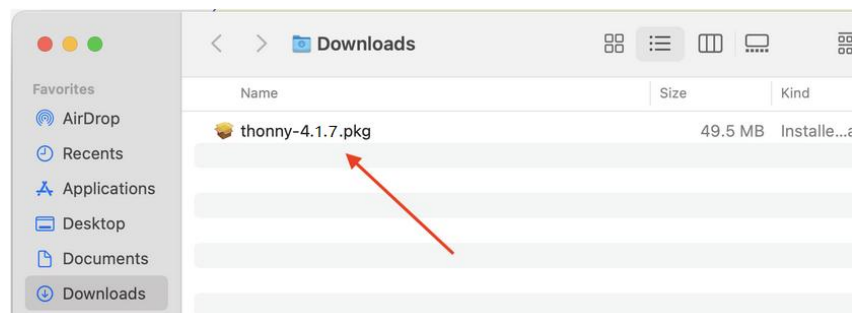
Esta sección de preguntas y respuestas recopila los **errores más frecuentes**, los **comportamientos inesperados** y los **problemas de funcionamiento** con los que se encuentran los principiantes al programar el ESP32-C3 con el IDE Thonny en Windows. Cada sección incluye los **síntomas del error**, las **causas fundamentales**, **soluciones paso a paso** y una descripción práctica de casos de error con capturas de pantalla (para facilitar su identificación). Todas las operaciones están pensadas para principiantes y se ajustan a los flujos de trabajo de desarrollo de MicroPython.

3.1 ¿Cómo instalar el IDE Thonny en macOS?

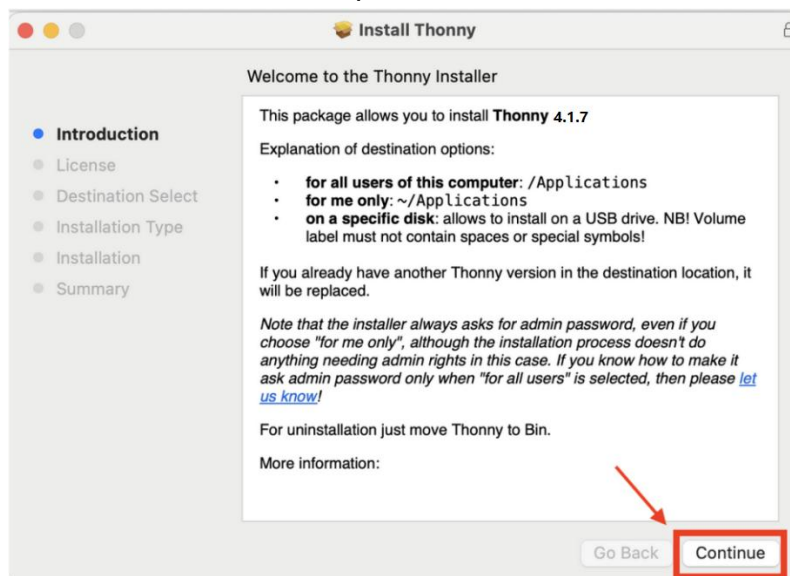
- 1) Accede a la página web oficial de Thonny: <https://thonny.org/>
- 2) Haz clic para descargar el instalador **thonny-4.1.7.pkg (42 MB)**.



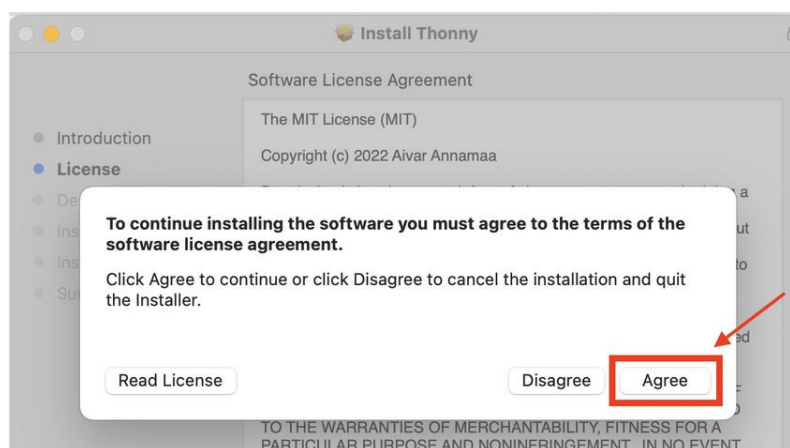
- 3) Abre y ejecuta el instalador y haz clic en «Permitir».



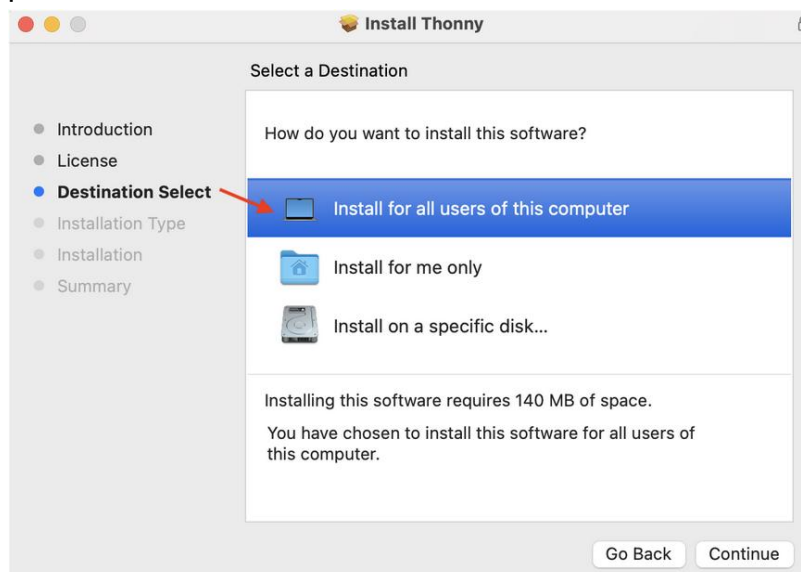
4) Haz clic en «Continuar» para confirmar la instalación



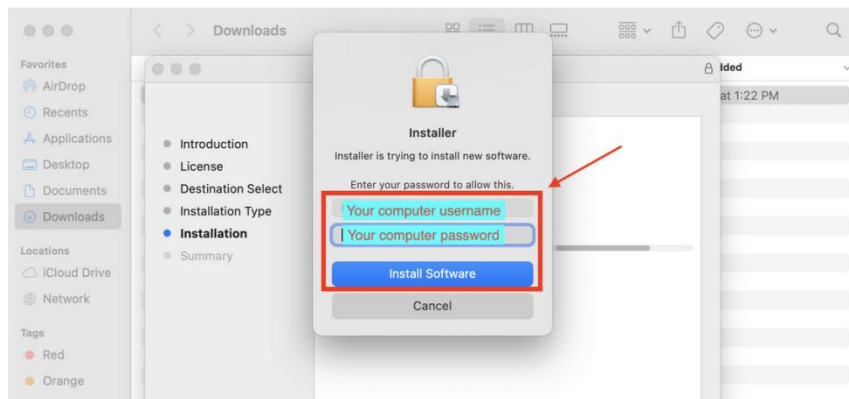
5) Sigue haciendo clic en «Continuar» y haz clic en «Aceptar» para aceptar los acuerdos de licencia



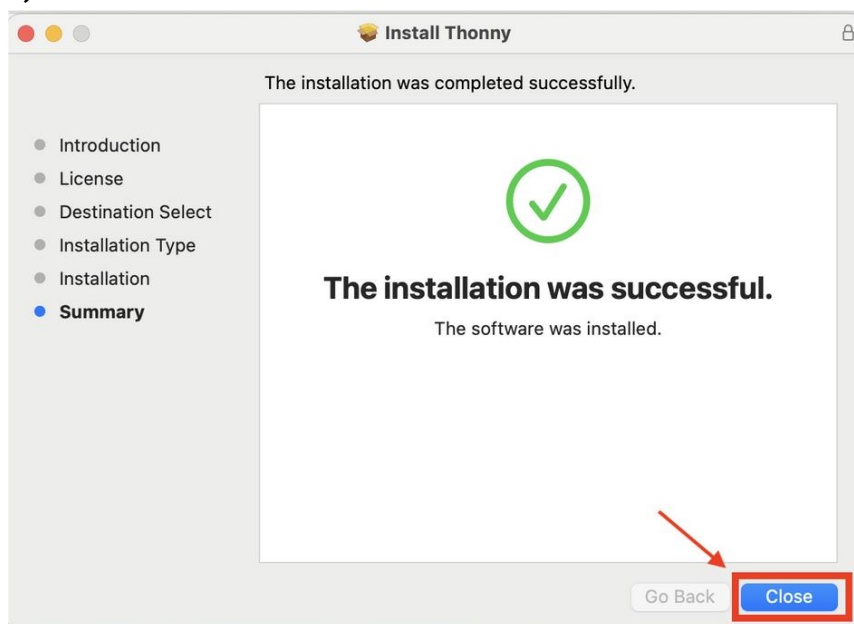
6) «Selecciona una ubicación» para la instalación. Puedes elegir la opción predeterminada.



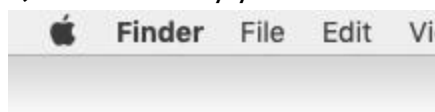
7) Haz clic en «Instalar» y en «Aceptar» para permitir que el instalador acceda a las carpetas, si es necesario. Deja que la instalación se complete.



8) Haz clic en «Cerrar»



9) El IDE Thonny ya está instalado y puedes hacer doble clic para abrirlo:



3.2 El brazo robótico no funciona

- 👉 ¿Debes activar el interruptor de encendido durante la instalación para inicializar el ángulo del servo?
- 👉 ¿Ha comprobado si el montaje es correcto?
- 👉 Compruebe si la batería tiene suficiente carga.
- 👉 Compruebe si la batería utilizada cumple con las especificaciones.

3.3 Errores de conexión de puertos y reconocimiento de dispositivos

P1: El menú de puertos situado en la esquina inferior derecha de Thonny no incluye la opción «MicroPython (ESP32)».

Síntomas del error

- En la esquina inferior derecha del IDE de Thonny solo aparece «Local Python 3.x», sin la opción **MicroPython (ESP32)**.
- Al abrir **Herramientas → Opciones → Intérprete → MicroPython (ESP32)**, no aparece la selección de puerto «USB Serial @ COMx» en el menú desplegable «Puerto o WebREPL».

Causas fundamentales

1. Falta el controlador **CH340 USB-a-serie** (las placas de desarrollo ESP32-C3 utilizan este chip para la comunicación USB; Windows no lo tiene preinstalado);
2. Cable USB suelto (se está utilizando un cable solo de carga en lugar de un cable de datos);
3. El ESP32-C3 no está encendido (el USB no está conectado o el interruptor de encendido de la placa está apagado);
4. Puerto USB dañado (los puertos USB del panel frontal de los ordenadores de sobremesa suelen ser inestables).

Soluciones paso a paso

1. **Instala el controlador USB a serie** (paso fundamental):
 - Sigue las instrucciones [de1.3](#) para instalarlo
2. **Comprueba el cable USB y el puerto:**
 - Sustitúyelo por un **cable USB de datos** (los cables de solo carga no tienen pines de datos y no pueden transmitir datos en serie);
 - Conecta el cable USB a un **puerto USB del panel trasero** (ordenador de sobremesa) o a un puerto USB directo del portátil (evita los concentradores USB);
3. **Comprueba el puerto en el Administrador de dispositivos de Windows:**
 - Pulsa **Win + X** → selecciona «**Administrador de dispositivos**» → despliega la sección «**Puertos (COM y LPT)**»;
 - Si se reconoce el ESP32-C3, verás «**USB-SERIAL CH340 (COMx)**» (x = un número como 3, 4 o 5);
 - Si aparece un signo de exclamación amarillo en el puerto, haz clic con el botón derecho → **Desinstalar dispositivo** → vuelve a conectar el cable USB para reinstalar el controlador.

P2: Thonny muestra el mensaje «No se ha podido abrir el puerto 'COMx'» al hacer clic en STOP

Síntomas del error

- Al hacer clic en el botón «**STOP**» de Thonny, el Shell muestra el error «No se ha podido abrir el puerto 'COM3'».
- El puerto aparece en el Administrador de dispositivos, pero no se puede utilizar en Thonny.

Causas fundamentales

1. El puerto serie está ocupado por **otro software** (por ejemplo, el IDE de Arduino, el monitor serie, Putty o los complementos serie de VS Code);
2. La conexión serie anterior de Thonny no se cerró correctamente (problema de caché del IDE);

3. Un proceso en segundo plano de Windows tiene ocupado el puerto serie.

Soluciones paso a paso

1. **Cierra todo el software relacionado con el puerto serie:**
 - Cierra el IDE de Arduino, el Monitor de serie, Putty y cualquier otra herramienta que pueda utilizar el puerto serie;
 - Comprueba si hay programas ocultos en segundo plano en la barra de tareas de Windows (haz clic con el botón derecho en la barra de tareas → Administrador de tareas → finaliza los procesos relacionados con las herramientas de serie).
2. **Reinicia el IDE de Thonny:**
 - Cierra Thonny por completo (haz clic en la «X» de la esquina superior derecha y asegúrate de que no haya ningún proceso de Thonny en el Administrador de tareas);
 - Vuelve a abrir Thonny y vuelve a seleccionar el puerto ESP32-C3.
3. **Vuelve a conectar el cable USB del ESP32-C3:**
 - Desconecta el cable USB, espera 2 segundos y vuelve a conectarlo; esto reinicia el puerto serie y libera la ocupación.

3.4 Errores de configuración del firmware y del intérprete de MicroPython

P1: Thonny muestra el mensaje «El dispositivo está ocupado o no responde» al ejecutar el código

Síntomas del error

- Haz clic en el botón «Run\STOP». Mensaje del shell: «El dispositivo está ocupado o no responde».

Causas fundamentales

1. El ESP32-C3 no tiene instalado el firmware de MicroPython (las placas nuevas suelen venir sin firmware preinstalado).

2. Se ha seleccionado un intérprete incorrecto en Thonny (por ejemplo, «MicroPython (ESP8266)» en lugar de «MicroPython (ESP32)»).
3. El firmware de MicroPython de la placa está dañado (debido a una actualización incompleta o a un corte de corriente durante la instalación del firmware).

Soluciones paso a paso

1. Actualiza el firmware en Thonny:

- i. Consulta la sección 4.5 para volver a flashear el firmware (no desconectes el cable USB durante el proceso).
- ii. Una vez finalizado, haz clic en **«Cerrar»** y reinicia el ESP32-C3 (pulsas el botón RESET).

2. Comprueba la instalación del firmware:

- Abre el terminal Shell de Thonny (**Ver → Shell**);
- Pulsa el botón RESET del ESP32-C3: verás la información de la versión de MicroPython y el indicador `>>>` (operación correcta).

P2: Thonny muestra el mensaje «Modelo de placa incorrecto» durante la instalación del firmware

Síntomas del error

- Al instalar el firmware de MicroPython, aparece un mensaje de error emergente: **«Se ha seleccionado un modelo de placa incorrecto. El firmware no es compatible con ESP32-C3»**;
- La actualización del firmware falla con un «Error de verificación» o un «Error de escritura».

Descripción de la captura de pantalla del caso de error

La barra de progreso de la instalación del firmware se detiene entre el 10 % y el 50 %, y aparece una advertencia en rojo: «El firmware para ESP32 (genérico) no es compatible con este dispositivo. Selecciona ESP32-C3 como modelo de placa».

Causa principal

Se ha seleccionado un **modelo de placa** incorrecto en la página de instalación del firmware (por ejemplo, «ESP32 (genérico)» en lugar de «ESP32-C3»): el ESP32-C3 utiliza un núcleo RISC-V, y el firmware genérico del ESP32 (núcleo Xtensa) es incompatible.

Solución paso a paso

1. Consulte la sección 4.5 para volver a flashear el firmware (no desconecte el cable USB durante el proceso de flasheo).

3.5 Errores de tiempo de ejecución (ejecución de código en ESP32)

P1: El código muestra «ImportError: no module named 'xxx'» al ejecutarse en el ESP32-C3

Síntomas del error

- Al hacer clic en «Ejecutar» o al reiniciar el ESP32-C3, aparece un error en rojo en la terminal REPL: **«ImportError: No module named 'led'»**.
- El código principal (por ejemplo, `main.py`) no se ejecuta porque no puede importar un módulo personalizado.

Captura de pantalla del caso de error Descripción

El shell muestra:

Texto sin formato

```
>>> import led
```

```
ImportError: No existe ningún módulo llamado «led»
```

```
>>> main.py:1: ImportError: No existe ningún módulo llamado «sensor»
```

Causas fundamentales

Caso 1: Módulo personalizado (p. ej., led.py, sensor.py) → No se ha subido al ESP32-C3

- El archivo del módulo importado solo se encuentra en el ordenador local con Windows, no en el sistema de archivos del ESP32-C3.

Caso 2: Módulo personalizado → Discrepancia en las mayúsculas y minúsculas del nombre de archivo

- MicroPython distingue **entre mayúsculas y minúsculas** en los nombres de archivo (p. ej., Led.py ≠ led.py, SENSOR.py ≠ sensor.py).

Soluciones paso a paso

1. **Para módulos personalizados: Sube el archivo del módulo al ESP32-C3**
 - En Thonny, abre el archivo del módulo que falta (p. ej., led.py);
 - Haz clic en **Archivo** → **Subir a/** para subirlo al ESP32-C3;
 - Comprueba que el archivo aparece en el panel «Archivos» de Thonny (debajo del dispositivo MicroPython);
 - Vuelve a ejecutar el código principal o reinicia la placa.
2. **Para módulos personalizados: corrige la discrepancia en las mayúsculas y minúsculas del nombre del archivo**

- Asegúrate de que el **nombre del archivo de importación coincida exactamente con el nombre real del archivo** (mayúsculas, minúsculas y ortografía);

Ejemplo: si el archivo se llama LedBlink.py, la importación debe ser `import LedBlink` (no `import ledblink`);

- Cambia el nombre del archivo a **todo en minúsculas** (recomendado para principiantes) para evitar errores de mayúsculas y minúsculas (p. ej., ledblink.py).

P2: El código muestra «OSError: [Errno 2] ENOENT» al ejecutarse en ESP3 2

Síntomas del error

- La terminal de shell muestra «**OSError: [Errno 2] ENOENT**» al ejecutar el código;
- Este error suele producirse al realizar operaciones con archivos (por ejemplo, al abrir un archivo) o al importar módulos.

Descripción de la captura de pantalla del caso de error

Texto sin formato

```
>>> f = open("data.txt", "r")
OSError: [Errno 2] ENOENT
>>> from led import run
OSError: [Errno 2] ENOENT
```

Causa principal

ENOENT = Error NO ENTity → El archivo o módulo al que el código intenta acceder **no existe** en el sistema de archivos del ESP32 (el error más común entre los principiantes).

- Para operaciones con archivos: el archivo de destino (por ejemplo, `data.txt`) no se ha cargado en la placa;
- Para importaciones: Falta el archivo del módulo (p. ej., `led.py`) o la ruta es incorrecta.

Soluciones paso a paso

1. Comprueba que el archivo o módulo existe en el ESP32-C3:

- Comprueba el panel «Archivos» de Thonny para confirmar que el archivo (por ejemplo, `data.txt`, `led.py`) está presente en el dispositivo MicroPython;
- Si no está, sube el archivo a la placa.

2. Corrige la ruta del archivo en el código:

- Utiliza **solo el nombre del archivo** para los archivos del directorio raíz (sin rutas absolutas o relativas como `/flash/led.py` o `./led.py`);

Correcto: `import led / f = open("data.txt", "r");`

Incorrecto: `import flash.led / f = open("./data.txt", "r").`

3. Comprueba que no haya errores ortográficos en el nombre del archivo:

- Asegúrate de que el nombre del archivo en el código coincide con el nombre real del archivo (sin espacios de más ni letras erróneas);

Ejemplo: `data.txt` ≠ `data .txt`, `led.py` ≠ `ledd.py`.

Pregunta 3: El código se ejecuta en Thonny (botón «Run»), pero no tras reiniciar el ESP32

Síntomas del error

- Al hacer clic en el botón «Run» de Thonny, el zumbador del ESP32 suena con normalidad;
- Tras reiniciar la placa (desconectar el USB o pulsar de nuevo RESET), el zumbador deja de emitir sonido (el código no se ejecuta automáticamente).

Descripción de la captura de pantalla del caso de error

El terminal de la shell muestra que el código se ejecuta correctamente al hacer clic en «Ejecutar»: `>>> # buzzer...;` tras el reinicio, el terminal solo muestra el indicador de versión de MicroPython y no se ejecuta ningún código.

Causas fundamentales

1. El código no se ha guardado como «`main.py`» en el ESP32-C3 (MicroPython ejecuta «`main.py`» automáticamente al arrancar);
2. `main.py` está guardado en el ordenador local con Windows, pero no se ha subido a la placa;
3. `main.py` contiene errores de sintaxis (lo que provoca que la ejecución automática falle al arrancar).

Soluciones paso a paso

1. Asegúrate de que el código de entrada se guarde como `main.py` y se suba a la placa (solución principal):
 - Si utilizas código de un solo archivo: guarda el código como `main.py` (Archivo → Guardar como) → Súbelo al ESP32 (Archivo → Subir a);

- Si utilizas código modular: Carga todos los módulos personalizados (por ejemplo, `buzzer.py`) → Crea un archivo `main.py` mínimo (por ejemplo, `import buzzer; buzzer.run()`) → Carga `main.py` en la placa.
- 2. **Comprueba que `main.py` se encuentre en el directorio raíz del ESP32:**
 - Comprueba el panel «Archivos» de Thonny para confirmar que `main.py` aparece en el dispositivo MicroPython (sin subcarpetas);
 - Si no aparece, vuelve a subir `main.py`.
- 3. **Comprueba si hay errores de sintaxis en `main.py`:**
 - En Thonny, abre el archivo `main.py` de la placa (haz doble clic en el panel «Archivo»);
 - Busca los **subrayados rojos** (comprobación sintáctica de Thonny) → corrige los errores (por ejemplo, faltan dos puntos `:`, errores de sangría, números de pin incorrectos);
 - Vuelve a cargar el archivo `main.py` corregido y reinicia la placa.
- 4. **Prueba la ejecución automática:**
 - Tras cargar el archivo `main.py` correcto, reinicia el ESP32 → el código se ejecuta automáticamente (zumbador) sin necesidad de hacer clic en el botón «Ejecutar» de Thonny.

3.6 Anomalías en el sistema de archivos y la visualización del dispositivo

P1: ¿El panel de archivos de Thonny no muestra «MicroPython Device»?

Captura de pantalla del caso de error Descripción

El panel de archivos de Thonny muestra la entrada del dispositivo MicroPython, pero el panel derecho muestra un texto en rojo: «No se han podido listar los archivos del dispositivo. Comprueba la conexión e inténtalo de nuevo».

Causas fundamentales

1. Este fenómeno se produce cada vez que se conecta el puerto USB del ordenador a la placa de control, ya que Thonny carece de la función de detección automática para la conexión de dispositivos ESP32.
2. La comunicación por el puerto serie es inestable (cable USB suelto, puerto USB defectuoso).

Soluciones paso a paso

1. Para que Thonny reconozca el dispositivo ESP32:

- Haz clic en el botón **«STOP»** o vuelve a seleccionar **«MicroPython (ESP32) ▪ USB Serial @ COMx»** en la esquina inferior derecha de Thonny.

2. Estabilizar la conexión serie:

- Cambia el cable de datos USB → conéctalo a un puerto USB trasero del ordenador de sobremesa (evita los concentradores);
- Asegúrate de que el cable USB no esté suelto (que el LED de la placa no parpadee de forma intermitente).

3.7 Problemas de ocupación del puerto serie y de pérdida de conexión

P1: El ESP32-C3 se desconecta aleatoriamente de Thonny (el puerto COM desaparece)

Descripción de la captura de pantalla del caso de error

En la esquina inferior derecha de Thonny, la indicación cambia repentinamente de «MicroPython en COM3» a «Python 3.x local»; en la sección «Puertos» del Administrador de dispositivos ya no aparece el puerto USB-SERIAL CH340.

Causas fundamentales

1. **Inestabilidad en el suministro de alimentación USB** (el portátil o el concentrador USB no proporcionan suficiente energía al ESP32);

2. Chip USB-serie dañado en la placa del ESP32 (problema de hardware);
3. Conflicto de controladores USB de Windows (controlador CH340 obsoleto);

Soluciones paso a paso

1. **Mejora la alimentación USB** (solución más habitual):
 - Para ordenadores portátiles: conecta el cable USB a un **concentrador USB con alimentación** (o conecta el portátil a un adaptador de corriente);
 - Para ordenadores de sobremesa: utiliza siempre **los puertos USB del panel trasero** (proporcionan una alimentación más estable que los del panel frontal);
 - Evita utilizar concentradores USB sin alimentación (no pueden suministrar suficiente corriente para el ESP32-C3).

4

Ponte en contacto con nosotros

Si no has encontrado una solución entre las anteriores, ponte en contacto con nuestro equipo de asistencia.

Para que podamos atenderte con mayor rapidez, ten a mano la siguiente información:

Su número de pedido.

El modelo del producto (por ejemplo, kit de brazo robótico E1R0000) y la versión del software

Una descripción detallada del problema o de su consulta.

Los pasos que ya ha intentado.

Cualquier captura de pantalla, foto o fragmento de código relevante relacionado con el mensaje de error.

¿Alguna otra consulta?

No dudes en ponerte en contacto con nosotros para:

Errores en los tutoriales y comentarios: ayúdanos a mejorar nuestra documentación.

Ideas y sugerencias sobre el producto: nos encantaría conocer tus ideas.

Alianzas y colaboración: ¿Te interesa trabajar con nosotros? Hablemos.

Descuentos y promociones: Solicita información sobre precios para el sector educativo o por volumen.

Cualquier otra cosa: para cualquier otra pregunta no técnica.



support@siyeenove.com



<https://siyeenove.com>